# Документация GPTHub v2.0.0

## Часть 1: Архитектурный обзор и структура проекта

### 1. Общая архитектура

GPTHub состоит из двух основных компонентов, работающих в связке:

1. **Форк OpenWebUI (`open-webui/`)** — основной веб-интерфейс для пользователя (порт 3000). В него интегрирована панель GPTHub Workspace (`/workspace/gpthub`) и добавлен прокси-слой для обращения к Pipelines.
2. **GPTHub Pipelines (`pipelines/` и корень проекта)** — "мозг" системы. Python-сервис (FastAPI, порт 9099), который обрабатывает запросы к AI-моделям, реализует "умный" роутер, управляет памятью и всей git-подобной системой ветвления чатов.

Оба компонента запускаются в Docker-контейнерах через `docker compose`. Все внутренние коммуникации между сервисами идут через Docker-сеть `gpthub_net`.

**Схема взаимодействия:**

graph TD Browser[Браузер] -->|:3000| OWU[OpenWebUI (форк)] OWU -->|/ollama| Pipelines OWU -->|/v1/chat/completions| Pipelines OWU -->|/api/v1/workspace/*| Pipelines subgraph Docker Network (gpthub_net) Pipelines -->|9099| MWS[MWS API] Pipelines -->|5432| Postgres[(PostgreSQL)] Pipelines -->|8000| ChromaDB[(ChromaDB)] Pipelines -->|8080| SearXNG[SearXNG] end Pipelines --> SQLite[(SQLite: chat_branches.db)]

### 2. Полное дерево проекта

```
C:\Users\Zeta\source\repos\task-repo\
├── open-webui/                    # Форк OpenWebUI (v0.8.12)
│   ├── Dockerfile                 # [ИЗМЕНЁН] Сборка OWU с нашим форком
│   ├── build/                     # [ИГНОР] Статика после npm run build
│   └── src/                       # [ИЗМЕНЁН] Исходники SvelteKit
│       ├── hooks.server.ts        # [ДОБАВЛЕН] Проксирование запросов на пайплайны
│       ├── lib/
│       │   ├── apis/
│       │   │   └── workspace.ts   # [ДОБАВЛЕН] API-клиент для Workspace
│       │   ├── components/
│       │   │   ├── layout/
│       │   │   │   └── Sidebar.svelte # [ИЗМЕНЁН] Добавлена кнопка "GPTHub Workspace"
│       │   │   └── workspace/     # [ДОБАВЛЕН] Новые компоненты GPTHub
│       │   │       ├── ChatDetail.svelte
│       │   │       ├── CommitList.svelte
│       │   │       ├── ProfileView.svelte
│       │   │       ├── ToastContainer.svelte
│       │   │       ├── WorkspacePanel.svelte
│       │   │       └── WorkspaceSidebar.svelte
│       │   └── stores/
│       │       └── workspace.ts   # [ДОБАВЛЕН] Сторы состояния Workspace
│       └── routes/
│           └── workspace/
│               └── gpthub/        # [ДОБАВЛЕН] Роут для GPTHub Workspace
│                   ├── +layout.svelte
│                   └── +page.svelte
│
├── build.ps1                      # Скрипт сборки для Windows
├── build.sh                       # Скрипт сборки для Linux/macOS
├── docker-compose.yml             # Оркестрация всех сервисов
├── Dockerfile.pipelines           # Сборка Python-сервиса пайплайнов
├── .env.example                   # Пример переменных окружения
│
├── pipelines/                     # GPTHub Pipelines (Python)
│   ├── main.py                    # Точка входа FastAPI
│   ├── config.py                  # Управление конфигурацией из ENV
│   ├── requirements.txt           # Python-зависимости
│   ├── router/
│   │   └── smart_router.py        # "Умный" роутер — ядро системы
│   ├── pipelines/                 # Реализации конкретных пайплайнов
│   │   ├── base.py                # Базовый класс для всех пайплайнов
│   │   ├── asr.py                 # Распознавание речи (Whisper)
│   │   ├── vlm.py                 # Анализ изображений (Vision)
│   │   ├── rag_file.py            # Ответы по документам (RAG)
│   │   ├── text_default.py        # Обычные текстовые запросы
│   │   ├── web_parse.py           # Чтение веб-страниц
│   │   ├── web_search.py          # Поиск в интернете
│   │   ├── code.py                # Генерация и анализ кода
│   │   ├── deep_research.py       # Глубокое исследование темы
│   │   ├── presentation.py        # Генерация .pptx презентаций
│   │   ├── image_gen.py           # Генерация изображений (DALL-E)
│   │   └── chat_git.py            # [КЛЮЧЕВОЙ] Обработчик git-команд
│   ├── memory/
│   │   └── manager.py             # Долгосрочная память пользователей
│   ├── chat_branches/             # Git-подобная система
│   │   ├── storage.py             # Работа с SQLite (ветки, коммиты, коды)
│   │   └── context.py             # Сжатие и управление контекстом
│   └── static/
│       └── workspace.html         # Альтернативная веб-панель (чистый HTML/JS)
│
└── searxng/
    └── settings.yml               # Конфигурация поискового движка
```

### 3. Описание ключевых файлов и их роли

**Корень проекта**

| Файл | Назначение |
|---|---|
| `docker-compose.yml` | Определяет все сервисы: `openwebui`, `pipelines`, `chromadb`, `searxng`, `postgresql`. Здесь же настраиваются переменные окружения и монтируются volume'ы для сохранения данных. |
| `build.ps1` / `build.sh` | Скрипты для упрощения сборки. Они выполняют `npm run build` внутри `open-webui/` и запускают `docker compose build`. |
| `.env.example` | Содержит все возможные переменные окружения для настройки системы. Нужно скопировать в `.env` и заполнить `MWS_API_KEY`. |
| `Dockerfile.pipelines` | Собирает образ для Python-сервиса. Устанавливает зависимости из `pipelines/requirements.txt` и копирует код. |

**Форк OpenWebUI (`open-webui/`)**

| Файл | Назначение |
|---|---|
| `Dockerfile` | **[ИЗМЕНЁН]** Сборка контейнера. Использует уже готовую статику из папки `build`, которая копируется внутрь контейнера. Это **необходимое условие**: фронтенд должен быть собран локально перед запуском `docker compose build`. |
| `src/hooks.server.ts` | **[ДОБАВЛЕН]** Серверный хук SvelteKit. Перехватывает запросы к `/api/v1/workspace/*` и проксирует их на `pipelines:9099`. Это нужно, чтобы фронтенд на порту `:3000` мог общаться с пайплайнами на порту `:9099` без проблем с CORS. |
| `src/lib/apis/workspace.ts` | **[ДОБАВЛЕН]** Типизированный API-клиент для общения с GPTHub. Содержит все интерфейсы (`Chat`, `Commit`, `Workspace`, ...) и функции для запросов. |
| `src/lib/components/workspace/*` | **[ДОБАВЛЕН]** Набор Svelte-компонентов, из которых состоит интерфейс GPTHub Workspace. |
| `src/lib/stores/workspace.ts` | **[ДОБАВЛЕН]** Управление реактивным состоянием всего Workspace UI (режимы, данные, уведомления). |
| `src/routes/workspace/gpthub/+page.svelte` | **[ДОБАВЛЕН]** Страница, которая собирает все компоненты воедино. Доступна по адресу `http://localhost:3000/workspace/gpthub`. |

**GPTHub Pipelines (`pipelines/`)**

| Файл | Назначение |
|---|---|
| `main.py` | Точка входа FastAPI-приложения. Определяет все эндпоинты: OpenAI-совместимые (`/v1/chat/completions`, `/v1/models`), специфичные для GPTHub (`/api/v1/gpthub/*`) и эндпоинты для Workspace API. |
| `config.py` | Загружает и валидирует все настройки из переменных окружения. Предоставляет единый объект конфигурации для всего приложения. |
| `smart_router.py` | **Сердце системы**. Анализирует входящий запрос и определяет, какой тип задачи перед ним (код, картинка, поиск, git-команда и т.д.) и какую модель использовать. |
| `pipelines/chat_git.py` | Обрабатывает все текстовые команды git-системы: `/share`, `/fork`, `/log`, `/branch` и т.д. Парсит команды и вызывает соответствующие методы из `storage.py`. |
| `chat_branches/storage.py` | **Хранилище git-системы**. Вся работа с SQLite-базой данных `chat_branches.db`. Здесь хранятся коммиты, share-коды, workspaces, участники. |
| `chat_branches/context.py` | Отвечает за "сжатие" контекста. Получает большой диалог и с помощью LLM создаёт краткое, но ёмкое резюме (summary) для сохранения в историю. |
| `memory/manager.py` | Управляет долгосрочной памятью о пользователе. Сохраняет и извлекает факты из векторной базы данных ChromaDB. |
| `static/workspace.html` | Альтернативный, "чистый" веб-интерфейс для отладки и использования без UI OpenWebUI. Работает напрямую с эндпоинтами пайплайнов. |

Отлично, продолжаем. После того как мы разобрались с общей архитектурой и расположением файлов, переходим к следующей критически важной секции — **конфигурации и запуску**.

Здесь я опишу все переменные окружения, что они делают и почему они важны, а также объясню двухэтапный процесс сборки, который является частой причиной проблем.

## Часть 2: Конфигурация и запуск

### 1. Переменные окружения (`.env`)

Файл `.env` — это главный пульт управления GPTHub. Он должен находиться в корне проекта (рядом с `docker-compose.yml`). Все сервисы читают свои настройки из этого файла.

#### Основные переменные (MWS API)

| Переменная | Обязательность | Значение по умолчанию | Описание |
|---|---|---|---|
| `MWS_API_KEY` | **Да** | *Нет* | Ключ API для доступа к моделям MWS. **Без него ничего не заработает**. |
| `MWS_API_BASE_URL` | Нет | `https://api.gpt.mws.ru` | Базовый URL для API MWS. |

#### Модели (MWS)

Эти переменные позволяют гибко настроить, какие модели будут использоваться для разных задач. Роутер (`smart_router.py`) опирается на эти значения.

| Переменная | Значение по умолчанию | Описание (для какой задачи используется) |
|---|---|---|
| `MWS_MODEL_DEFAULT` | `qwen2.5-72b-instruct` | Основная модель для большинства текстовых задач. |
| `MWS_MODEL_LONG` | `gpt-oss-120b` | Модель с большим контекстным окном (для длинных диалогов и `deep_research`). |
| `MWS_MODEL_CODE` | `qwen3-coder-480b-a35b` | Модель для генерации и анализа кода. |
| `MWS_MODEL_VLM` | `cotype-pro-vl-32b` | Основная модель для анализа изображений (Vision Language Model). |
| `MWS_MODEL_VLM_FALLBACK` | `qwen2.5-vl-72b` | Резервная VLM-модель. |
| `MWS_MODEL_ASR` | `whisper-turbo-local` | Модель для распознавания речи (Audio Speech Recognition). |
| `MWS_MODEL_EMBEDDINGS` | `bge-m3` | Модель для создания векторных представлений текста (эмбеддингов). Используется для памяти и RAG. |
| `MWS_MODEL_IMAGE` | `qwen-image-lightning` | Основная модель для генерации изображений. |
| `MWS_MODEL_IMAGE_FALLBACK` | `qwen-image` | Резервная модель для генерации изображений. |
| `MWS_MODEL_SAFE_FALLBACK` | `mws-gpt-alpha` | Самая надёжная и всегда доступная модель. Используется как "последняя надежда", если все остальные недоступны. |

#### Базы данных и сервисы

| Переменная | Значение по умолчанию | Описание |
|---|---|---|
| `POSTGRES_PASSWORD` | `gpthub` | Пароль для PostgreSQL. Используется OpenWebUI для хранения пользователей и чатов. |
| `WEBUI_SECRET_KEY` | `gpthub_change_me_in_prod` | Секретный ключ для шифрования сессий OpenWebUI. **Настоятельно рекомендуется сменить в production**. |
| `SEARXNG_SECRET_KEY` | `gpthub_searxng_secret` | Секретный ключ для SearXNG. |
| `CHROMADB_HOST` | `chromadb` | Имя хоста ChromaDB (внутри Docker-сети). |
| `CHROMADB_PORT` | `8000` | Порт ChromaDB. |
| `SEARXNG_URL` | `http://searxng:8080` | URL для доступа к SearXNG из контейнера с пайплайнами. |

#### Поведение роутера и пайплайнов

| Переменная | Значение по умолчанию | Описание |
|---|---|---|
| `MEMORY_TOP_K` | `3` | Количество фактов из памяти, которое будет добавлено в контекст LLM. |
| `MEMORY_SIMILARITY_THRESHOLD` | `0.75` | Порог косинусного сходства для извлечения фактов из памяти (от 0 до 1). Выше — строже. |
| `ROUTER_LONG_CTX_TOKENS` | `6000` | Порог в токенах, после которого роутер переключится на модель с длинным контекстом (`MWS_MODEL_LONG`). |
| `WEB_PARSE_TIMEOUT` | `10` | Таймаут в секундах для загрузки и парсинга веб-страницы. |
| `IMAGE_GEN_TIMEOUT` | `60` | Таймаут в секундах для генерации одного изображения. |

## 2. Процесс сборки и запуска

Запуск GPTHub — это **двухэтапный процесс**, и это важно понимать.

**Почему два этапа?** OpenWebUI состоит из двух частей: фронтенд (SvelteKit) и бэкенд (Python). Чтобы не тянуть тяжелые Node.js-зависимости и не собирать фронтенд внутри Docker-контейнера (что медленно и создает большой образ), сборка фронтенда вынесена на хост-машину. Docker только копирует уже готовые файлы.

#### Этап 1: Сборка фронтенда OpenWebUI

Этот этап выполняется на вашей локальной машине (Windows/Linux) **один раз** при первой установке и каждый раз после изменения файлов в форке OpenWebUI (например, если вы правите UI).

Команды:

- **Windows (PowerShell):** `.\build.ps1`
- **Linux / macOS (Bash):** `chmod +x build.sh && ./build.sh`

**Что делают эти скрипты?**

1. Переходят в папку `open-webui/`.
2. Выполняют `npm ci --force`, чтобы установить все зависимости из `package-lock.json`.
3. Выполняют `npm run build`, чтобы скомпилировать SvelteKit-приложение в папку `open-webui/build/`.
4. Возвращаются в корень и запускают `docker compose build` и `docker compose up -d`.

**Этап 2: Сборка и запуск Docker-контейнеров**

`docker compose build` собирает три основных образа:

1. `gpthub_openwebui`: Берет за основу Python-образ, копирует папку `open-webui/build/` (собранную на Этапе 1), папку `backend/` и запускает веб-сервер.
2. `gpthub_pipelines`: Собирается из `Dockerfile.pipelines` в корне. Устанавливает Python-зависимости и копирует всю папку `pipelines/`.
3. Остальные сервисы (`chromadb`, `searxng`, `postgresql`) скачиваются готовыми из Docker Hub.

`docker compose up -d` запускает все контейнеры в фоне.

### 3. Проверка работоспособности

После успешного запуска должны быть доступны следующие URL:

- **OpenWebUI:** http://localhost:3000 (http://localhost:3000)
  - Логин: `admin@gpthub.local`
  - Пароль: `gpthub2024`
- **GPTHub Workspace (интегрированный):** http://localhost:3000/workspace/gpthub (http://localhost:3000/workspace/gpthub)
- **GPTHub Pipelines (API):** http://localhost:9099/health (http://localhost:9099/health)
- **Альтернативная веб-панель:** http://localhost:9099/workspace (http://localhost:9099/workspace)

### 4. Устранение типичных проблем при запуске

| Проблема | Вероятная причина и решение |
|---|---|
| При запуске `docker compose up` выходит ошибка, что образ `openwebui` не найден. | **Не выполнен Этап 1.** Нужно сначала запустить `.\build.ps1` (или `./build.sh`), чтобы собрать папку `build/`. Docker не может собрать контейнер без нее. |
| Ошибка "MWS_API_KEY не задан". | В корне проекта нет файла `.env`. Скопируйте `.env.example` в `.env` и пропишите в нем ваш ключ `MWS_API_KEY`. |
| В GPTHub Workspace не отображаются мои чаты. | Вы вошли в OpenWebUI под пользователем `admin@gpthub.local`. Нужно использовать "реального" пользователя. В пайплайнах нет авторизации, поэтому **любой, кто знает User ID, может получить доступ к чатам этого пользователя** (это важное допущение для MVP). |
| Команды `/share`, `/fork` не работают в чате OpenWebUI. | Возможны две причины:<br>1. В `docker-compose.yml` не добавлена переменная `PIPELINES_URL=http://pipelines:9099` для сервиса `openwebui`. Без неё бэкенд OWU не знает, куда проксировать запросы.<br>2. Не применен патч для бэкенда OpenWebUI, который добавляет проксирование. (Проверь, что `open-webui/backend/open_webui/main.py` содержит соответствующий `mount`). |

## Часть 3: Система плагинов (Pipelines) — ядро обработки запросов

Этот слой отвечает за всю "магию" GPTHub: от приёма запроса от OpenWebUI до выбора нужной модели, выполнения задачи и сохранения результатов. Он построен как модульная система, где каждый тип задачи (код, картинка, поиск) обрабатывается своим специализированным пайплайном.

### 1. Обзор слоя Pipelines

`pipelines/` — это независимое FastAPI-приложение. Оно полностью отделено от OpenWebUI и общается с ним только по HTTP. Такая архитектура позволяет обновлять или перезапускать пайплайны, не трогая основной интерфейс, и наоборот.

**Жизненный цикл одного запроса:**

1. OpenWebUI отправляет POST-запрос на `/v1/chat/completions` (или другой эндпоинт) на `http://pipelines:9099`.
2. FastAPI-приложение (`main.py`) принимает запрос.
3. Запрос передаётся в **`SmartRouter`**, главный диспетчер.
4. **`TaskDetector`** (внутри `SmartRouter`) анализирует текст, файлы и историю, чтобы понять, что от него хотят.

5. `SmartRouter` выбирает один из **специализированных пайплайнов** (например, `CodePipeline`, `ImageGenPipeline` или `ChatGitPipeline`).
6. Выбранный пайплайн выполняет работу: вызывает MWS API, парсит сайты, ищет в памяти, генерирует изображения и т.д.
7. Результат (обычно в виде потока чанков) возвращается в OpenWebUI и отображается пользователю.
8. В фоне (`fire-and-forget`) пайплайн может сохранить информацию о диалоге в **память** (`MemoryManager`) или в **git-систему** (`BranchStorage`).

## 2. Точка входа и инфраструктура: `main.py` и `config.py`

### `main.py`

Этот файл создаёт и настраивает FastAPI-приложение.

**Ключевые эндпоинты:**

- **OpenAI-совместимые (основные):**
  - `POST /v1/chat/completions`: Главный эндпоинт для общения с моделями.
  - `GET /v1/models`: Возвращает список доступных моделей (`gpthub-auto` + все модели из MWS API).
  - `POST /v1/audio/transcriptions`: Транскрипция аудио (для голосового ввода).
  - `POST /v1/embeddings`: Получение векторных представлений текста (для RAG в OWU).
  - `POST /v1/audio/speech`: TTS (синтез речи). **Реализована заглушка**, возвращающая тишину, так как MWS не предоставляет TTS-модель.
- **Специфичные для GPTHub:**
  - `GET /api/v1/gpthub/health`: Расширенная проверка здоровья, которая также пингует MWS API, ChromaDB и SearXNG.
  - `POST /api/v1/gpthub/router/diagnose`: Эндпоинт для диагностики роутера. Позволяет отправить текст и посмотреть, какой `TaskType` и модель он выберет.
  - `GET /api/v1/gpthub/tools`: Возвращает статус и конфигурацию инструментов (поиск, парсинг).
- **Git-система (Workspace API):**
  - `GET /api/v1/workspace/chats`: Список чатов пользователя.
  - `GET /api/v1/workspace/chat/{chat_id}/log`: История коммитов чата (с пагинацией).
  - `POST /api/v1/workspace/join`, `/invite`, `/members`: Управление Workspaces.
  - `GET /workspace`: Отдаёт альтернативную HTML-панель `workspace.html`.

**Управление жизненным циклом (`lifespan`):** В `main.py` реализован асинхронный контекстный менеджер `lifespan`. Это правильный способ инициализировать и корректно завершать ресурсы в FastAPI.

- **Startup:** Создаётся экземпляр `SmartRouter` и вызывается `await router.setup()`. **Это критически важно**. `setup()` инициализирует все 11 пайплайнов, которые, в свою очередь, подключаются к `_SharedResources`. Без этого вызова приложение запустится, но при первом же запросе упадёт с ошибкой.
- **Shutdown:** Вызывает `await _SharedResources.close()`, что корректно закрывает HTTP-клиенты и пулы потоков, предотвращая утечки ресурсов.

### `config.py`

Этот файл — единый источник правды для всех настроек пайплайнов.

**Проблема, которую он решает:** Раньше значения по умолчанию и чтение из `os.environ` были разбросаны по десяткам файлов. Это приводило к дублированию, несогласованности и трудностям в тестировании.

**Решение:**

- **`_Defaults`**: Внутренний класс, в котором **централизованно** хранятся все дефолтные значения. Одно место для правки.
- **`MWSConfig`, `ServicesConfig`, `RouterConfig`**: `frozen` датаклассы. Они неизменяемы после создания, что исключает случайное изменение конфигурации во время работы.
- **Валидация**: При загрузке конфигурация проходит проверку (например, что порт в диапазоне 1-65535). Все предупреждения логируются, но не останавливают запуск.
- **Синглтон `_ConfigRegistry`**: Гарантирует, что конфигурация будет загружена только один раз. Метод `reset()` используется в тестах для перезагрузки с новыми переменными окружения.

## 3. Базовый класс и разделяемые ресурсы: `base.py`

Это фундамент, на котором построены все пайплайны.

### `_SharedResources`

**Проблема, которую он решает:** Каждый пайплайн создаётся заново для обработки **каждого** входящего запроса. Если бы каждый пайплайн создавал свои собственные `httpx.AsyncClient` или `AsyncOpenAI`, это привело бы к:

- **Утечке соединений:** Открывалось бы огромное количество TCP-соединений к MWS API, ChromaDB и т.д.
- **Истощению ресурсов:** Закончились бы файловые дескрипторы или порты.
- **Плохой производительности:** Постоянное пересоздание клиентов и установка новых соединений — дорогая операция.

**Решение:** `_SharedResources` — это синглтон, который **один раз** создаёт и хранит экземпляры:

- `openai`: `AsyncOpenAI` клиент для MWS API.
- `http`: `httpx.AsyncClient` с настроенными пулами соединений.
- `memory`: `MemoryManager` для доступа к ChromaDB.

Все пайплайны получают доступ к этим объектам через `self.client`, `self.http`, `self.memory`. Это гарантирует, что соединения переиспользуются, а ресурсы расходуются экономно.

#### `BasePipeline`

Абстрактный класс, который наследуют все пайплайны.

**Ключевые методы:**

- `async def setup()`: Инициализирует пайплайн, подключая его к `_SharedResources`. Вызывается один раз при старте приложения.
- `async def run(self, body: dict) -> AsyncGenerator[str, None]`: **Абстрактный метод**, который **обязан** реализовать каждый дочерний пайплайн. Содержит всю логику обработки запроса. Возвращает асинхронный генератор, чтобы можно было стримить ответ по мере генерации.
- `def fire_and_forget(self, coro)`: **Критически важный метод**. Позволяет запустить фоновую задачу (например, сохранение в память), не дожидаясь её завершения и не блокируя основной поток ответа пользователю. При этом гарантируется, что все ошибки в фоновой задаче будут корректно залогированы, а не "проглочены".
- `async def stream_llm(...)`: Универсальная функция для выполнения стримингового запроса к LLM. **Автоматически обрабатывает fallback на резервную модель**, если основная недоступна, и имеет встроенную защиту от "зависания" модели с помощью `asyncio.timeout`.

## 4. "Умный" роутер: `smart_router.py`

Это диспетчер, который решает, какой пайплайн и какую модель использовать для входящего запроса.

#### `TaskDetector.detect()`

Этот метод содержит всю логику определения типа задачи (`TaskType`). Он работает по строгой системе приоритетов, что позволяет избежать конфликтов.

**Приоритеты детекции (от высшего к низшему):**

1. **Git-команды чата (`/share`, `/fork`, `/branch` ...):** `TaskType.CHAT_GIT`. **Самое важное правило**. Git-команды всегда обрабатываются в первую очередь, чтобы они не были приняты за что-то другое.
2. **Системные запросы OpenWebUI:** (например, "Generate a title", "### Task:"). Определяются по длине (>2000 символов) или по наличию ключевых маркеров. Для них принудительно используется безопасная fallback-модель, чтобы не тратить ресурсы и не выполнять поиск в интернете.
3. **Явные команды (`/research`, `/present`, `/image`):** Если запрос начинается с этих команд, роутер сразу направляет его в соответствующий пайплайн.
4. **Ручной выбор модели пользователем:** Если пользователь в интерфейсе явно выбрал модель (не "gpthub-auto"), роутер использует её и не выполняет автоматическое определение.
5. **Файлы (аудио, изображения, документы):** Определяются по MIME-типу или расширению. Если прикреплён файл, задача однозначно направляется в `ASR`, `VLM` или `RAG_FILE`.
6. **URL в сообщении:** Если в тексте есть ссылка, это задача `WEB_PARSE`.
7. **Генерация изображений:** Если в тексте есть ключевые слова "нарисуй", "сгенерируй", "create image" и т.д.
8. **Веб-поиск:** Если есть ключевые слова "найди", "поищи", "search" и т.д.
9. **Код:** Определяется по фразам ("напиши код"), техническим терминам ("алгоритм", "рефакторинг") или синтаксическим паттернам (```, `def `).
10. **Длинный контекст:** Если история диалога превышает `ROUTER_LONG_CTX_TOKENS`, выбирается модель для длинного контекста.
11. **`TEXT_DEFAULT`:** Если ничего из вышеперечисленного не подошло, используется стандартный текстовый пайплайн.

#### `SmartRouter.route()`

После того, как `TaskDetector` определил тип задачи, `route()` выбирает нужный пайплайн из словаря `self._pipelines` и вызывает его метод `run()`.

**Важные детали:**

- В `body["_gpthub"]` добавляется метаинформация: результат детекции, `user_id`, `chat_id`. Это нужно, чтобы пайплайны могли получить доступ к этим данным через `self.get_meta(body)`.

- **Обработка ошибок:** Если внутри пайплайна происходит необработанное исключение, `route()` перехватывает его и возвращает пользователю стандартное сообщение об ошибке, не "роняя" всё приложение.

# Часть 4: Специализированные пайплайны (Pipelines)

Каждый пайплайн наследуется от `BasePipeline` и реализует свой метод `run()`. При создании они получают доступ к `_SharedResources` (через `self.client`, `self.http`, `self.memory`), что позволяет им вызывать MWS API, ходить в интернет и работать с памятью, не создавая новых соединений.

## 1. Простые текстовые пайплайны: `text_default.py` и `code.py`

Это два базовых пайплайна, которые просто берут историю сообщений, обогащают её (опционально) и отправляют в LLM.

#### `TextDefaultPipeline` (`text_default.py`)

**Назначение:** Универсальный пайплайн для большинства обычных текстовых запросов.

**Алгоритм работы:**

1. Получает последнее сообщение пользователя и его `user_id`.
2. **Обращается к долгосрочной памяти** (`self.memory.retrieve()`). Получает список фактов о пользователе, релевантных текущему запросу.
3. Если факты есть, **динамически добавляет их в `system_prompt`**. Это ключевой механизм персонализации: модель "вспоминает", кто перед ней, и адаптирует ответ.
4. Отправляет обогащенный запрос в LLM и стримит ответ пользователю.
5. В фоне (`fire_and_forget`) **сохраняет состоявшийся диалог обратно в память** (`self.memory.save()`) и в git-систему (`self.save_chat_context()`).

#### `CodePipeline` (`code.py`)

**Назначение:** Специализированная обработка задач, связанных с программированием.

**Отличия от `TextDefaultPipeline`:**

- **Более строгий `system_prompt`:** Требует отвечать кодом, оформленным в markdown-блоки, с комментариями.
- **Температура модели (`temperature`):** Жёстко задана как `0.2` для большей детерминированности и точности кода.
- **Фильтрация фактов из памяти:** Перед добавлением в контекст, все факты из памяти проходят через `_filter_tech_facts()`. Этот фильтр оставляет **только технические факты** (языки, фреймворки, инструменты), отбрасывая личную информацию (например, "любит кошек"). Это гарантирует, что контекст для написания кода будет чистым и релевантным.

## 2. Пайплайны для работы с файлами и медиа: `asr.py`, `vlm.py`, `rag_file.py`

Эти пайплайны активируются, когда пользователь прикрепляет к запросу файл.

#### `ASRPipeline` (`asr.py`) — Распознавание речи

**Назначение:** Преобразование аудиофайла в текст, а затем ответ на вопрос на основе этого текста.

**Алгоритм работы:**

1. **Валидация:** Проверяет, что прикреплённый файл — это аудио (`_is_audio_file`).
2. **Декодирование:** Извлекает base64 данные из запроса, обрабатывая разные форматы (data URL, чистый base64).
3. **Транскрипция (Whisper):** Отправляет аудио в MWS Whisper API (`self.client.audio.transcriptions.create`). Используется `asyncio.wait_for` с таймаутом `_WHISPER_TIMEOUT_SEC`, чтобы не зависнуть навсегда.
4. **Формирование нового запроса:** Если транскрипция успешна, создаётся новый запрос к LLM, в котором к истории диалога добавляется сообщение: `[Транскрипт аудио]: {result.text}`.
5. **Стриминг:** Полученный текст отправляется в `TextDefaultPipeline` (через `self.stream_llm`), и ответ стримится пользователю.

**Обработка ошибок:** Вместо падения с исключением, пайплайн возвращает понятное пользователю сообщение об ошибке (`_TranscribeFail` и `_format_error`).

#### `VLMPipeline` (`vlm.py`) — Анализ изображений

**Назначение:** Ответ на вопрос, подкреплённый одним или несколькими изображениями.

**Алгоритм работы:**

1. **Подготовка изображений:** Из `files` в `body` извлекаются изображения и преобразуются в формат data URL (`...`), который принимает Vision API.

2. **Сборка мультимодального запроса:** Строится специальное `user`-сообщение, в котором `content` — это массив из текстового запроса и объектов `image_url`.
3. **Запрос к VLM:** Используется `self.client.chat.completions.create`. Поскольку не все модели поддерживают vision, реализована **цепочка fallback-моделей** (`_build_model_chain`). Сначала запрос идёт к `MWS_MODEL_VLM`, а в случае неудачи — к `MWS_MODEL_VLM_FALLBACK`.
4. **Стриминг ответа.**

#### `RAGFilePipeline` (`rag_file.py`) — Ответы по документам

**Назначение:** Ответы на вопросы по содержимому загруженного файла (PDF, DOCX, TXT).

**Алгоритм работы:**

1. **Извлечение текста:** В зависимости от MIME-типа файла, используется `pypdf`, `python-docx` или простое чтение текста. Это делается в синхронных функциях, но вызывается через `run_in_executor`, чтобы не блокировать event loop.
2. **Чанкинг:** Извлечённый текст разбивается на перекрывающиеся фрагменты (чанки) по ~1500 символов. Перекрытие нужно, чтобы не терять контекст на границах чанков.
3. **Индексация в ChromaDB:**
   - Для всех чанков создаются эмбеддинги с помощью модели `MWS_MODEL_EMBEDDINGS`.
   - Чанки и их эмбеддинги сохраняются во **временную** коллекцию ChromaDB. Имя коллекции содержит `session_id`, поэтому у каждого запроса своя коллекция. Это позволяет избежать конфликтов.
4. **Семантический поиск:** Вопрос пользователя также векторизуется, и в ChromaDB выполняется поиск наиболее релевантных чанков (косинусное сходство).
5. **Формирование контекста:** Найденные релевантные чанки добавляются в системный промпт LLM как "контекст из файла".
6. **Стриминг ответа.**

**Fallback-стратегия:** Если ChromaDB недоступен, пайплайн не падает, а использует **первые N чанков** из документа в качестве контекста. Это менее точно, но позволяет системе продолжить работу.

## 3. Пайплайны для работы с интернетом: `web_search.py` и `web_parse.py`

#### `WebSearchPipeline` (`web_search.py`)

**Назначение:** Поиск актуальной информации в интернете через SearXNG.

**Ключевые особенности:**

- **Защита от системных запросов:** Функция `_is_system_request()` гарантирует, что служебные промпты OpenWebUI не будут отправлены в поиск.
- **Дедупликация:** Результаты от SearXNG могут содержать ссылки на один и тот же сайт. Функция `_deduplicate_results()` оставляет только один результат с каждого домена, чтобы контекст был разнообразным.
- **Retry-логика:** Если первый запрос к SearXNG не удался, делается ещё одна попытка с небольшой паузой.
- **Сборка контекста:** Результаты форматируются и вставляются в начало истории диалога как системное сообщение.

#### `WebParsePipeline` (`web_parse.py`)

**Назначение:** Извлечение и анализ содержимого конкретной веб-страницы по URL.

**Алгоритм работы:**

1. **Извлечение URL:** Из текста запроса с помощью регулярного выражения извлекается первая ссылка.
2. **Загрузка страницы:** Через `self.http` делается GET-запрос.
   - **Проверка Content-Type:** Если сервер возвращает не HTML/текст, а PDF или изображение, запрос прерывается с ошибкой `NOT_TEXT`.
   - **Ограничение размера:** Если страница больше `_MAX_HTML_BYTES` (5MB), загрузка останавливается.
3. **Извлечение текста:**
   - **Приоритетный способ:** Используется `BeautifulSoup` для извлечения текста из семантических тегов (`<p>`, `<h1>` и т.д.).
   - **Fallback:** Если `BeautifulSoup` не установлен, используется грубый, но надёжный regex-парсер `_extract_text_regex`.
4. **Сборка контекста:** Извлечённый текст (обрезанный до `_MAX_CONTENT_CHARS`) добавляется в начало истории как системное сообщение.
5. **Стриминг ответа.**

## 4. Сложные генеративные пайплайны: `image_gen.py`, `deep_research.py`, `presentation.py`

Эти пайплайны выполняют тяжелые задачи, состоящие из нескольких шагов.

#### `ImageGenPipeline` (`image_gen.py`)

**Алгоритм:**

1. **Улучшение промпта:** Пользовательский запрос (например, "кот") отправляется в LLM с промптом `_ENHANCE_PROMPT`, который просит модель сделать его более детальным и художественным (например, "photorealistic portrait of a cat, cinematic lighting, 8k").
2. **Генерация:** Улучшенный промпт отправляется в модель генерации изображений (через `self.client.images.generate`). Реализована цепочка fallback-моделей (`_build_model_chain`).
3. **Отображение:** Пользователю показывается сгенерированное изображение и улучшенный промпт.

#### `DeepResearchPipeline` (`deep_research.py`)

Самый сложный пайплайн, выполняющий полноценное исследование.

**Алгоритм:**

1. **Декомпозиция:** Тема исследования отправляется в LLM, который разбивает её на `_SUBTOPICS_COUNT` подзапросов.
2. **Поиск:** Для каждого подзапроса параллельно выполняется поиск через `WebSearchPipeline`.
3. **Анализ страниц:** Для найденных URL параллельно загружаются страницы, из текста извлекается суть (с помощью LLM) и проверяется на релевантность теме.
4. **Синтез отчёта:** Все проанализированные фрагменты собираются в один большой промпт и отправляются в LLM (используя модель с длинным контекстом), который генерирует итоговый структурированный отчёт.
5. **Стриминг:** Отчёт стримится пользователю по мере генерации.

#### `PresentationPipeline` (`presentation.py`)

Генерирует `.pptx` файл на лету.

**Алгоритм:**

1. **Генерация структуры:** Тема отправляется в LLM с жёстким промптом, требующим вернуть JSON с описанием слайдов (заголовок, тип, пункты).
2. **Парсинг и валидация:** JSON парсится и проверяется на минимальные требования (например, не слишком ли короткие пункты).
3. **Рендеринг PPTX:** В функции `_build_pptx`, которая выполняется в отдельном потоке (`run_in_executor`), с помощью библиотеки `python-pptx` создаётся презентация.
4. **Отдача файла:** Пользователю возвращается `data URL` с готовым `.pptx` файлом, который можно сразу скачать.

### 5. Git-пайплайн: `chat_git.py`

Этот пайплайн — "командный центр" для всей git-подобной системы. Он **не генерирует ответы с помощью LLM**, а выполняет команды и возвращает текстовый результат.

**Алгоритм работы:**

1. Парсит входящий текст запроса. Если он начинается с известной команды (`/share`, `/fork`, `/log`, `/branch`), то:
2. Вызывает соответствующий метод-обработчик (например, `_handle_share`).
3. Метод-обработчик парсит аргументы команды, вызывает нужные функции из `BranchStorage` и `ContextManager` и формирует текстовый ответ.
4. Возвращает ответ пользователю.

**Примеры обработчиков:**

- `_handle_share()`: Создаёт share-код, вызывает `storage.create_share_code()`.
- `_handle_fork()`: Получает контекст по коду (`context.get_context_for_fork()`), импортирует его (`context.import_context()`) и сохраняет связь в БД.
- `_handle_log()`: Запрашивает у `storage` историю коммитов и форматирует её в читаемый вид.

Этот пайплайн — ключевой элемент, связывающий пользовательские команды в чате с хранилищем `chat_branches.db`.

## Часть 5: Git-подобная система ветвления чатов

Эта система позволяет "коммитить" контекст диалогов, делиться им через "share-коды", "форкать" чужие чаты и работать в команде через "workspaces".

### 1. Обзор компонентов

Система состоит из трёх основных файлов, каждый из которых решает свою задачу:

| Файл | Назначение |
|---|---|
| `chat_branches/storage.py` | **Хранилище.** Низкоуровневая работа с SQLite-базой данных `chat_branches.db`. Все CRUD-операции над чатами, коммитами, share-кодами и workspaces. |
| `chat_branches/context.py` | **Сжатие и управление контекстом.** Отвечает за превращение длинных диалогов в краткие "резюме" (summary) с помощью LLM, а также за сборку контекста для форков. |

| Файл | Назначение |
| --- | --- |
| `memory/manager.py` | **Долгосрочная персональная память.** Хранит факты о пользователе в векторной базе данных ChromaDB и извлекает их по семантическому сходству. |

Взаимодействие между ними выглядит так:

graph TD subgraph "Запрос пользователя" User[Пользователь в чате] -->|"/fork ABC-123"| ChatGit[ChatGitPipeline] end subgraph "Git-система" ChatGit -->|get_share_code| Storage[BranchStorage (SQLite)] ChatGit -->|get_context_for_fork| CtxMgr[ContextManager] CtxMgr -->|get_commits| Storage CtxMgr -->|search_context| Chroma[(ChromaDB)] CtxMgr -->|LLM Call| MWS[MWS API] ChatGit -->|import_context| CtxMgr CtxMgr -->|add to Chroma| Chroma ChatGit -->|save_fork_link| Storage end subgraph "Фоновая работа" AnyPipeline[Любой пайплайн] -->|save_chat_context| BasePipeline BasePipeline -->|compress_and_store| CtxMgr CtxMgr -->|add to Chroma| Chroma CtxMgr -->|LLM Call| MWS CtxMgr -->|save_commit| Storage end

## 2. Хранилище: `chat_branches/storage.py`

Это бэкенд всей git-системы. Это синхронный код для работы с SQLite, но все публичные методы — асинхронные, так как они вызываются из асинхронного кода пайплайнов через `run_in_executor`.

**Структура базы данных (`chat_branches.db`)**

База данных спроектирована так, чтобы хранить всю необходимую информацию о ветвлении.

```
-- Workspaces (команды)
CREATE TABLE workspaces (
    id          TEXT PRIMARY KEY,
    name        TEXT NOT NULL,
    owner_id    TEXT NOT NULL,
    created_at  INTEGER NOT NULL
);

-- Участники workspaces
CREATE TABLE workspace_members (
    workspace_id  TEXT NOT NULL,
    user_id       TEXT NOT NULL,
    role          TEXT NOT NULL DEFAULT 'member',
    joined_at     INTEGER NOT NULL,
    invited_by    TEXT, -- кто пригласил
    PRIMARY KEY (workspace_id, user_id)
);

-- Invite-коды для вступления в workspace
CREATE TABLE workspace_invites (
    id            TEXT PRIMARY KEY,
    workspace_id  TEXT NOT NULL,
    invite_code   TEXT NOT NULL UNIQUE,
    created_by    TEXT NOT NULL,
    created_at    INTEGER NOT NULL,
    expires_at    INTEGER NOT NULL,
    max_uses      INTEGER NOT NULL DEFAULT 1,
    use_count     INTEGER NOT NULL DEFAULT 0
);

-- Реестр всех чатов
CREATE TABLE chat_contexts (
    chat_id       TEXT PRIMARY KEY,
    user_id       TEXT NOT NULL,
    workspace_id  TEXT,                -- связь с workspace
    is_public     INTEGER NOT NULL DEFAULT 0,
    created_at    INTEGER NOT NULL,
    updated_at    INTEGER NOT NULL
);

-- Коммиты — сохранённые "слепки" диалогов
CREATE TABLE context_commits (
    id                  TEXT PRIMARY KEY,
    chat_id             TEXT NOT NULL,
    user_id             TEXT NOT NULL,
    timestamp           INTEGER NOT NULL,
    pipeline_type       TEXT NOT NULL DEFAULT 'text', -- тип пайплайна
    user_message        TEXT NOT NULL,                -- вопрос пользователя
    assistant_summary   TEXT NOT NULL,                -- сжатое резюме ответа
    tags                TEXT NOT NULL DEFAULT '[]',   -- теги (JSON)
    tokens_count        INTEGER NOT NULL DEFAULT 0,
    parent_id           TEXT,                         -- для cherry-pick
    FOREIGN KEY (chat_id) REFERENCES chat_contexts(chat_id)
);

-- Share-коды для доступа к контексту
CREATE TABLE share_codes (
    code            TEXT PRIMARY KEY,
    chat_id         TEXT NOT NULL,
    created_by      TEXT NOT NULL,
    created_at      INTEGER NOT NULL,
    expires_at      INTEGER NOT NULL,
    is_permanent    INTEGER NOT NULL DEFAULT 0,
```

```
    alias           TEXT UNIQUE,          -- кастомное имя кода
    use_count       INTEGER NOT NULL DEFAULT 0,
    used_by         TEXT,                 -- кто использовал (для одноразовых)
    used_at         INTEGER,
    commit_ids      TEXT NOT NULL DEFAULT '[]' -- JSON-массив ID коммитов
);

-- Связи форков (кто у кого форкнул)
CREATE TABLE fork_links (
    child_chat_id   TEXT NOT NULL, -- кто форкнул
    parent_chat_id  TEXT NOT NULL, -- у кого форкнули
    share_code      TEXT NOT NULL,
    forked_at       INTEGER NOT NULL,
    PRIMARY KEY (child_chat_id, parent_chat_id)
);
```

#### Ключевые классы и методы `BranchStorage`

Все публичные методы — асинхронные, но внутри вызывают синхронные "близнецы" с суффиксом `_sync` через `run_in_executor`. Это сделано для того, чтобы не блокировать основной event loop пайплайнов при работе с файловой системой SQLite.

**Работа с Share-кодами:**

- `create_share_code()`: Создаёт новый код. Может быть одноразовым (48 часов) или постоянным (`is_permanent`). Поддерживает `alias` (кастомное имя). **Валидирует alias** через `_validate_alias()` и имеет **rate-limit** на создание постоянных кодов (`_PERMANENT_RATE_LIMIT`).
- `get_share_code()`: Проверяет, существует ли код и не истёк ли его срок.
- `mark_code_used()`: Увеличивает счётчик использований. Для одноразовых кодов также записывает, кто и когда использовал.
- `revoke_share_code()`: Удаляет код из базы. Только создатель может отозвать свой код.

**Работа с коммитами:**

- `save_commit()`: Сохраняет новый "слепок" диалога в `context_commits`. Сюда попадает уже сжатое резюме от `ContextManager`.
- `get_commits()`: Возвращает историю коммитов для чата, отсортированную от новых к старым.
- `get_commits_page()`: То же самое, но с пагинацией для UI.

**Работа с Workspaces:**

- `create_workspace()`: Создаёт workspace и сразу делает создателя его `owner`.
- `join_workspace()`: Добавляет пользователя в workspace по invite-коду. Проверяет срок действия и лимит использований.
- `create_invite()`: Создаёт invite-код. Проверяет, что вызывающий — `owner` или `admin`.
- `get_workspace_members()`, `remove_workspace_member()`: Управление участниками.

## 3. Сжатие и управление контекстом: `chat_branches/context.py`

Этот модуль решает самую сложную задачу: как превратить длинный диалог с моделью в компактное, но информативное резюме, которое можно сохранить в историю и использовать позже.

#### `ContextManager.compress_and_store()`

Этот метод вызывается **в фоне** (`fire_and_forget`) после каждого ответа модели (из любого пайплайна через `BasePipeline.save_chat_context()`).

**Алгоритм работы:**

1. **Выбор промпта:** На основе `pipeline_type` (например, "code", "web_search", "image_gen") выбирается один из специализированных промптов сжатия (`_TEXT_COMPRESS_PROMPT`, `_CODE_COMPRESS_PROMPT` и т.д.). Это **критически важно**, так как резюме для кода должно выглядеть иначе, чем для поиска в интернете.
2. **Проверка на "SKIP":** Все промпты начинаются с инструкции `_SKIP_INSTRUCTION`. Она учит LLM отвечать `SKIP`, если диалог не несёт полезной нагрузки (например, "Привет. — Привет! Чем могу помочь?"). Это позволяет не засорять историю пустыми разговорами.
3. **Вызов LLM:** Модели (`MWS_MODEL_SAFE_FALLBACK`) отправляется вопрос пользователя и ответ ассистента, и она генерирует краткое резюме (summary).
4. **Генерация тегов:** Параллельно вызывается `_TAG_PROMPT`, чтобы получить 2-4 ключевых слова (тега) для этого диалога.
5. **Сохранение:**
   - **В ChromaDB:** Summary сохраняется в коллекцию `branch_{chat_id}`. Это нужно для семантического поиска по истории внутри одного чата.
   - **В SQLite:** Через `BranchStorage.save_commit()` сохраняется запись в таблицу `context_commits`.
6. Возвращает `(summary, tags)`.

#### `ContextManager.get_context_for_fork()`

Этот метод вызывается, когда пользователь применяет `/fork ABC-123`.

#### Алгоритм работы:

1. **Чтение из SQLite:** Получает **все** коммиты исходного чата через `BranchStorage.get_commits()`.
2. **Фильтрация:** Если передан `commit_ids` (например, при `cherry-pick`), оставляет только указанные коммиты.
3. **Группировка:** Коммиты группируются по `pipeline_type`.
4. **Приоритизация:** Коммиты разных типов имеют разную важность. Например, `code`, `deep_research` и `presentation` идут первыми, так как несут больше технической ценности.
5. **Форматирование:** Каждый коммит форматируется с помощью `_format_commit_for_fork()`. Эта функция применяет **лимит символов в зависимости от типа** (`_CONTEXT_LIMITS`) и обрезает summary, если оно слишком длинное.
6. **Сборка:** Все отформатированные фрагменты собираются в один большой текст с разделителями `---`.
7. **Финальная обрезка:** Если итоговый текст превышает `_TOTAL_CONTEXT_LIMIT`, он обрезается.
8. **Возврат:** Собранный контекст возвращается в `ChatGitPipeline`, который затем импортирует его в целевой чат через `import_context()`.

### 4. Долгосрочная память: `memory/manager.py`

В отличие от `context.py`, который сжимает **конкретный диалог**, `MemoryManager` управляет **персональными фактами о пользователе** в долгосрочной перспективе.

#### Архитектура

`MemoryManager` построен по принципу разделения ответственности (SRP) и использует два внутренних класса-помощника:

- **`_ChromaGateway`:** Низкоуровневая работа с ChromaDB. Инкапсулирует в себе:
  - Управление клиентом (синглтон).
  - Кеширование коллекций.
  - Выполнение всех синхронных вызовов в `ThreadPoolExecutor`.
- **`_AIGateway`:** Вызовы к MWS API. Управляет `AsyncOpenAI` клиентом и выполняет эмбеддинги и запросы к LLM.

#### Как это работает?

##### Сохранение факта (`save()`):

1. Вызывается в фоне из любого пайплайна.
2. Проверяет, что сообщение не системное (`is_system_request`).
3. Отправляет сообщение пользователя в LLM с промптом `MEMORY_EXTRACTION_PROMPT`. LLM возвращает список фактов в формате `- Факт 1`.
4. Парсит ответ (`parse_facts`), фильтрует мусорные факты (`is_bad_fact`).
5. Для каждого валидного факта:
   - Создаёт его эмбеддинг через `_AIGateway.embed()`.
   - Проверяет, нет ли уже в ChromaDB очень похожего факта (`_DEDUP_SIMILARITY > 0.95`). Если есть — пропускает, чтобы не плодить дубликаты.
   - Сохраняет факт и его эмбеддинг в коллекцию `mem_{user_id}`.

##### Извлечение фактов (`retrieve()`):

1. Вызывается **синхронно** перед ответом модели (в `TextDefaultPipeline` и `CodePipeline`).
2. Векторизует текущий запрос пользователя.
3. Ищет в ChromaDB `top_k` ближайших фактов.
4. Отфильтровывает факты, чьё косинусное сходство ниже порога `MEMORY_SIMILARITY_THRESHOLD`.
5. Возвращает список найденных фактов для вставки в `system_prompt`.

## Часть 6: Фронтенд (UI/UX) — интеграция с OpenWebUI

Фронтенд GPTHub встроен прямо в интерфейс OpenWebUI как отдельная страница и не требует отдельного сервера. Он написан с использованием Svelte и TypeScript, повторяя технологический стек основного проекта.

### 1. Как это встроено?

Интеграция выполнена на нескольких уровнях:

1. **Роутинг:** В SvelteKit-приложение OpenWebUI добавлен новый маршрут — `/workspace/gpthub`. Все файлы для этого маршрута лежат в `src/routes/workspace/gpthub/`.

2. **Навигация:** В главное меню (сайдбар) OpenWebUI добавлена новая кнопка "□ GPTHub", которая ведёт на этот маршрут. Изменения внесены в файл `src/lib/components/layout/Sidebar.svelte`.
3. **Проксирование API:** Чтобы фронтенд на `:3000` мог общаться с пайплайнами на `:9099` и не сталкиваться с ошибками CORS, в `src/hooks.server.ts` добавлен серверный хук. Он перехватывает все запросы к `/api/v1/workspace/*` и `/api/v1/profile/*` и проксирует их на `PIPELINES_URL` (который в Docker-сети равен `http://pipelines:9099`).

## 2. Обзор файлов фронтенда

```
src/
├── routes/
│   └── workspace/
│       └── gpthub/
│           ├── +layout.svelte      # Макет страницы (почти пустой)
│           └── +page.svelte        # Главный компонент страницы
├── lib/
│   ├── apis/
│   │   └── workspace.ts            # API-клиент и TypeScript-типы
│   ├── stores/
│   │   └── workspace.ts            # Реактивное состояние (Svelte stores)
│   └── components/
│       └── workspace/              # UI-компоненты
│           ├── WorkspaceSidebar.svelte
│           ├── ChatDetail.svelte
│           ├── CommitList.svelte
│           ├── WorkspacePanel.svelte
│           ├── ProfileView.svelte
│           └── ToastContainer.svelte
```

## 3. API-клиент и типы: `workspace.ts`

Этот файл — мост между фронтендом и бэкендом.

- **TypeScript-интерфейсы:** Здесь строго описаны все типы данных, которые приходят с бэкенда: `Chat`, `Commit`, `Workspace`, `ShareCode` и т.д. Это даёт автодополнение в IDE и защищает от ошибок.
- **Функции-хелперы:** `apiFetch<T>()` — универсальная функция для выполнения HTTP-запросов. Она автоматически обрабатывает ошибки и парсит JSON.
- **Объект `workspaceApi`:** Содержит методы для вызова всех нужных эндпоинтов: `getChats()`, `getLog()`, `createInvite()`, `joinWorkspace()` и т.д. Каждый метод асинхронный и возвращает `Promise` с ожидаемым типом.

## 4. Управление состоянием: `stores/workspace.ts`

В Svelte для реактивности используются **сторы** (stores). Этот файл содержит все сторы, необходимые для работы Workspace UI.

**Основные сторы:**

- **Режимы отображения:** `workspaceMode` ('my', 'workspaces', 'profile').
- **Данные:** `myChats`, `myWorkspaces`, `profileChats`. Содержат массивы данных, загруженных с бэкенда.
- **Выбранные элементы:** `selectedChatId`, `selectedWorkspaceId`, `selectedProfileChatId`. Реагируя на их изменение, компоненты перерисовывают детали.
- **Пагинация:** `commitsPagination`, `profilePagination`. Хранят состояние пагинации для списков коммитов.
- **Уведомления:** `toasts`. Стор для глобальной очереди всплывающих уведомлений. Функция `showToast()` добавляет уведомление и автоматически удаляет его через 3.5 секунды.

## 5. UI-компоненты

Страница `/workspace/gpthub` собрана из нескольких независимых компонентов.

**`+page.svelte`**

**Роль:** Корневой компонент-сборщик.

**Что делает:**

1. При монтировании (`onMount`) вызывает `loadMyData()`, которая параллельно загружает чаты и workspaces пользователя через `workspaceApi` и сохраняет их в сторы `myChats` и `myWorkspaces`.
2. Рендерит общую структуру страницы:

    - Слева: `<WorkspaceSidebar />`
    - Справа (основная область): в зависимости от `$workspaceMode` показывает либо `<ChatDetail />`, либо `<WorkspacePanel />`, либо `<ProfileView />`.
3. Содержит `<ToastContainer />` для отображения уведомлений.

### `WorkspaceSidebar.svelte`

**Роль:** Навигация по разделам Workspace.

**Что делает:**

- Отображает три вкладки-режима: "Мои чаты", "Workspace", "Профиль". Переключение между ними меняет стору `workspaceMode`.
- **В режиме "Мои чаты":** Отображает список `$myChats`. При клике на чат устанавливает `selectedChatId`.
- **В режиме "Workspaces":** Отображает список `$myWorkspaces`. При клике на workspace устанавливает `selectedWorkspaceId`.
- **В режиме "Профиль":** Ничего не отображает, показывая заглушку.

### `ChatDetail.svelte`

**Роль:** Отображение детальной информации о выбранном чате.

**Что делает:**

- Реагирует на изменение `$selectedChatId`.
- Загружает и отображает:
    1. Историю коммитов (через `CommitList`).
    2. Список форков (через API `getForks`).
    3. Список активных share-кодов (через API `getCodes`).
- Предоставляет кнопки для копирования команд `/share` и `/log`.
- Позволяет отозвать share-код.

### `CommitList.svelte`

**Роль:** Универсальный компонент для отображения истории коммитов с пагинацией.

**Что делает:**

- Принимает на вход массив `commits`, состояние `loading`, объект `pagination`.
- Рендерит список коммитов, форматируя каждый с помощью `renderCommitItem`.
- Отображает панель пагинации (кнопки "Вперёд/Назад", номера страниц).
- **Важно:** Этот компонент используется и в `ChatDetail`, и в `ProfileView`, обеспечивая единый вид истории.

### `WorkspacePanel.svelte`

**Роль:** Управление workspaces.

**Что делает:**

- Показывает форму для вступления в workspace по `invite-коду`.
- Отображает детали выбранного workspace: название, ID, участников, инвайты.
- Предоставляет кнопки для создания новых инвайтов (одноразовых и мульти).
- Для `owner` и `admin` показывает кнопки управления участниками (например, удалить).

### `ProfileView.svelte`

**Роль:** Просмотр публичных профилей других пользователей.

**Что делает:**

- Содержит поле ввода `User ID`.
- После ввода ID загружает публичные чаты этого пользователя через `getPublicChats()`.
- Отображает список чатов в боковом сайдбаре.
- При выборе чата загружает его историю коммитов через `CommitList`.
- Предоставляет кнопку "□ Форкнуть", которая получает `share-код` для выбранного чата и копирует в буфер обмена команду `/fork ABC-123`.

Отлично. Продолжаю дописывать документацию, закрывая пробелы, которые сам же и нашёл. Я буду действовать по тому же принципу: разбирать каждый "проваленный" пункт с максимальной детализацией, объясняя не только "что", но и "как именно" и "почему так".

Я помню контекст всех файлов, которые ты мне предоставил. Продолжаем.

---

# Документация GPTHub v2.0.0 (Продолжение)

## Часть 7: Критические точки взаимодействия и "подводные камни"

В этой части я детально разберу те места, где система может вести себя неочевидно, а также ключевые механизмы, без понимания которых разработка будет затруднена.

### 1. Инъекция метаданных в `body` запроса: как пайплайны узнают `chat_id` и `user_id`

Это одна из самых важных и неочевидных точек во всей архитектуре.

**Где это происходит:** Файл `pipelines/router/smart_router.py`, метод `SmartRouter.route()`.

**Как это выглядит в коде:**

```
# pipelines/router/smart_router.py (фрагмент)

async def route(self, body: dict) -> AsyncGenerator[str, None]:
    # ...
    user_id = body.get("_user_id_resolved", "anonymous")
    # ...

    # Создаём объект с метаданными
    body["_gpthub"] = {
        "detection":         detection,
        "user_id":           user_id,
        "chat_id":           body.get("_user_info", {}).get("chat_id", ""),
        "last_user_message": last_user_message,
        "user_info":         body.get("_user_info", {}),
    }

    # Вызываем пайплайн с уже обогащённым body
    pipeline = self._pipelines.get(detection.task_type, ...)
    async for chunk in pipeline.run(body):
        yield chunk
```

**Почему это так важно:**

1. **До этого момента** `body` содержал только то, что пришло от OpenWebUI (сообщения, модель, файлы).
2. **После этого** в `body` появляется ключ `_gpthub`, который содержит **всю служебную информацию**, необходимую для сохранения контекста.
3. Любой пайплайн может получить эти данные через `self.get_meta(body)`, который определён в `base.py`:

```
# pipelines/pipelines/base.py (фрагмент)

class BasePipeline(ABC):
    # ...
    def get_meta(self, body: dict) -> RequestMeta:
        return RequestMeta(body.get("_gpthub", {}))
```

**Без этого механизма** ни один пайплайн не смог бы сохранять коммиты в git-систему, потому что они не знали бы `chat_id` и `user_id`.

---

### 2. Как на самом деле работает проксирование запросов к моделям

В документации я упомянул, что `hooks.server.ts` проксирует только Workspace API. Давай разберём **все слои проксирования**, чтобы не было путаницы.

**Слой 1: Проксирование Workspace API (фронтенд → пайплайны)**

- **Файл:** `open-webui/src/hooks.server.ts`

- **Что делает:** Перехватывает запросы от браузера к `/api/v1/workspace/*` и `/api/v1/profile/*` и перенаправляет их на `http://pipelines:9099`.
- **Зачем:** Чтобы Svelte-фронтенд на порту `:3000` мог общаться с пайплайнами на порту `:9099` без ошибок CORS.

**Слой 2: Проксирование запросов к AI-моделям (OpenWebUI → пайплайны)**

Этот слой **не** в `hooks.server.ts`. Он настраивается в **самом OpenWebUI** через переменные окружения.

- **Где настраивается:** `docker-compose.yml`, секция `openwebui`.
- **Ключевые переменные:**

```
environment:
  - OPENAI_API_BASE_URL=http://pipelines:9099/v1
  - OPENAI_API_KEY=gpthub-pipelines-key
```

- **Что делает:** Говорит бэкенду OpenWebUI (Python), что все запросы к OpenAI-совместимому API нужно отправлять не на `api.openai.com`, а на `http://pipelines:9099/v1`.

**Слой 3: Проксирование аудио, эмбеддингов и изображений**

Для некоторых специфичных функций OpenWebUI нужно явно указать, какой "движок" использовать.

- **Где настраивается:** `docker-compose.yml`, секция `openwebui`.
- **Ключевые переменные:**

```
environment:
  # RAG (загрузка документов)
  - RAG_EMBEDDING_ENGINE=openai
  - RAG_OPENAI_API_BASE_URL=http://pipelines:9099/v1
  - RAG_OPENAI_API_KEY=gpthub-pipelines-key

  # Генерация изображений
  - ENABLE_IMAGE_GENERATION=true
  - IMAGE_GENERATION_ENGINE=openai
  - IMAGES_OPENAI_API_BASE_URL=http://pipelines:9099/v1
  - IMAGES_OPENAI_API_KEY=gpthub-pipelines-key

  # Speech-to-Text (аудио в текст)
  - AUDIO_STT_ENGINE=openai
  - AUDIO_STT_OPENAI_API_BASE_URL=http://pipelines:9099/v1
  - AUDIO_STT_OPENAI_API_KEY=gpthub-pipelines-key
```

- **Что делает:** Когда пользователь загружает документ в OpenWebUI, система создаёт эмбеддинги. Эти переменные говорят ей: "Используй OpenAI-совместимый движок и отправляй запросы на `http://pipelines:9099/v1`".

**Схема всех трёх слоёв:**

```
Браузер (localhost:3000)
    |
    ├─ /api/v1/workspace/* ──► hooks.server.ts ──► pipelines:9099 (прямой прокси)
    |
    └─ /v1/chat/completions ──► бэкенд OpenWebUI ──► pipelines:9099/v1 (через OPENAI_API_BASE_URL)
```

---

## 3. Почему две базы данных: SQLite и ChromaDB

Это архитектурное решение, которое может показаться избыточным, но оно оправдано.

| Характеристика | SQLite (`context_commits`) | ChromaDB (`branch_{chat_id}`) |
|---|---|---|
| **Тип данных** | Структурированные записи (строки и колонки). | Векторные представления (эмбеддинги) текста. |
| **Способ поиска** | Точный поиск по ID, дате, типу. | Семантический поиск по смыслу ("найди похожее"). |
| **Использование в GPTHub** | 1. История коммитов (`/log`).<br>2. Формирование контекста для форка (`get_context_for_fork`). | 1. Семантический поиск по истории внутри чата (`search_context`).<br>2. **НЕ ИСПОЛЬЗУЕТСЯ** для форков. |
| **Почему не одна БД?** | SQLite не умеет искать "по смыслу". | ChromaDB не предназначена для хранения точной хронологии и метаданных. |

**Ключевой вывод:** `get_context_for_fork()` **читает только из SQLite**. Это гарантирует, что результат форка всегда будет **детерминированным и полным**, а не зависеть от капризов семантического поиска.

---

### 4. Как UI узнаёт, что данные изменились (реактивность Svelte)

В документации я описал сторы, но не объяснил, как они связаны с компонентами.

**Пример: Загрузка чатов.**

1. **Компонент `+page.svelte`** вызывает `await workspaceApi.getChats(userId)`.
2. Полученный массив чатов сохраняется в стору: `myChats.set(data.chats)`.
3. **Компонент `WorkspaceSidebar.svelte`** содержит строчку `$: console.log($myChats)`. Символ `$` перед именем стору автоматически подписывает компонент на изменения этой стору.
4. Как только в `+page.svelte` выполнится `myChats.set(...)`, **все компоненты, которые используют `$myChats`, будут мгновенно перерисованы** с новыми данными.

**Пример: Выбор чата.**

1. Пользователь кликает на чат в `WorkspaceSidebar.svelte`.
2. Выполняется `selectedChatId.set(chat.chat_id)`.
3. **Компонент `ChatDetail.svelte`** содержит реактивный блок:

```
$: if ($selectedChatId) {
    loadPage(0);
    loadForks();
    loadCodes();
}
```

4. Svelte видит, что `$selectedChatId` изменился, и **автоматически выполняет код внутри `$: if (...)`**, загружая данные для нового чата.

## Часть 8: Детальное форматирование контекста для форка (`_format_commit_for_fork`)

В документации я упомянул, что для разных типов пайплайнов контекст форматируется по-разному. Теперь разберём это с конкретными примерами из кода.

Файл `pipelines/chat_branches/context.py`, функция `_format_commit_for_fork()`.

**Входные данные:**

- `commit`: словарь с полями `pipeline_type`, `user_message`, `assistant_summary`.
- `char_limit`: максимальное количество символов для этого коммита (берётся из `_CONTEXT_LIMITS`).

**Логика для разных типов:**

**1. Тип `code`**

```
if ptype == "code":
    user_short = user_msg[:80] if user_msg else ""
    result = f"[КОД] {user_short}\n{summary}" if user_short else f"[КОД]\n{summary}"
```

**Пример результата:**

```
[КОД] напиши функцию сложения двух чисел
Язык: Python
Задача: Реализация функции сложения двух чисел
Реализовано:
  • add(a: int, b: int) int — возвращает сумму a и b
```

**2. Тип `deep_research`**

```
elif ptype == "deep_research":
    result = f"[ИССЛЕДОВАНИЕ]\n{summary}"
```

**Пример результата:**

```
[ИССЛЕДОВАНИЕ]
Тема: Влияние ИИ на рынок труда в 2025 году
Выводы:
  • Автоматизация затронет 30% рабочих мест в сфере обслуживания
  • Спрос на AI-специалистов вырос на 45% за последний год
  • Наиболее устойчивы творческие профессии и управление персоналом
```

**3. Тип `image_gen`**

```
elif ptype == "image_gen":
    result = f"[ИЗОБРАЖЕНИЕ]\n{summary}"
```

**Пример результата:**

```
[ИЗОБРАЖЕНИЕ]
Сгенерировано: фотorealistic portrait of a cat wearing a wizard hat, cinematic lighting, bokeh background, 8k, highly detailed.
```

**4. Тип `rag_file`**

```
elif ptype == "rag_file":
    file_hint = ""
    if "[Файл:" in user_msg:
        file_hint = user_msg.split("]")[0] + "]"
    result = f"[ДОКУМЕНТ {file_hint}]\n{summary}" if file_hint else f"[ДОКУМЕНТ]\n{summary}"
```

**Пример результата:**

```
[ДОКУМЕНТ [Файл: annual_report_2024.pdf]]
В отчёте указано, что выручка компании за 2024 год составила $12.4 млн, что на 15% выше показателя 2023 года.
Основные драйверы роста: запуск продукта X и выход на рынок Азии.
```

**Почему это важно:** Такое форматирование позволяет при форке **быстро понять контекст**, не читая весь сырой диалог. Получатель форка видит структурированную выжимку, адаптированную под тип контента.

Понял. Дополняю документацию в том порядке, который я сам же и наметил.

---

# Документация GPTHub v2.0.0 (Продолжение)

## Часть 9: Обработка ответов от MWS API и fallback-стратегии в `stream_llm`

Функция `stream_llm` в `base.py` — это единая точка для всех вызовов LLM. Понимание того, как она обрабатывает ошибки MWS API, критически важно для отладки проблем с моделями.

### 1. Формат ошибок MWS API

MWS API (как и большинство OpenAI-совместимых API) возвращает ошибки в следующем JSON-формате:

```
{
  "error": {
    "message": "The model `qwen2.5-72b-instruct` does not exist or you do not have access to it.",
    "type": "invalid_request_error",
    "param": null,
    "code": "model_not_found"
  }
}
```

Типичные HTTP-статусы:

- `400 Bad Request` — невалидный запрос (например, неподдерживаемый MIME-тип для VLM).
- `401 Unauthorized` — неверный или отсутствующий API-ключ.
- `404 Not Found` — запрошенная модель не существует.
- `429 Too Many Requests` — превышен rate-limit.
- `500 Internal Server Error` — внутренняя ошибка на стороне MWS.
- `503 Service Unavailable` — сервис временно недоступен (перегрузка или обслуживание).

## 2. Алгоритм работы `stream_llm` при ошибках

Файл `pipelines/pipelines/base.py`, метод `stream_llm`.

**Шаг 1: Формирование цепочки моделей**

```
fallback_chain = _build_fallback_chain(model, cfg.model_safe_fallback)
```

- Если основная модель и fallback-модель совпадают, цепочка состоит из одной модели.
- Если разные — из двух: `[model, cfg.model_safe_fallback]`.

**Шаг 2: Последовательный перебор моделей**

```
for idx, try_model in enumerate(fallback_chain):
    is_last = (idx == len(fallback_chain) - 1)

    try:
        async for chunk in self._stream_single_model(
            model=try_model,
            messages=full_messages,
            temperature=temperature,
            max_tokens=max_tokens,
        ):
            yield chunk
        return   # Успешно завершили стрим — выходим из всей функции
```

**Шаг 3: Обработка конкретных типов исключений**

```
    except asyncio.TimeoutError:
        last_error = asyncio.TimeoutError(f"Модель {try_model} не ответила вовремя")
        logger.warning("stream_llm: таймаут модели %s", try_model)

    except Exception as exc:
        last_error = exc
        logger.warning("stream_llm: модель %s недоступна: %s", try_model, exc, exc_info=True)
```

**Шаг 4: Сообщение пользователю и переход к следующей модели**

```
    # Сообщаем о переключении (только если есть следующая попытка)
    if not is_last:
        yield (
            f"\n⚠ Модель `{try_model}` недоступна, "
            f"переключаюсь на резервную...\n\n"
        )
```

**Шаг 5: Все модели исчерпаны — финальная ошибка**

```
# После цикла, если ни одна модель не сработала
logger.error(
    "stream_llm: все модели недоступны, chain=%s, last_error=%s",
    fallback_chain,
    last_error,
    exc_info=last_error,
)
yield (
    "\n\n **Сервис временно недоступен.**\n"
    "Попробуйте повторить запрос через несколько минут.\n"
)
```

### 3. Таймауты внутри `_stream_single_model`

Функция `_stream_single_model` использует **двухуровневую систему таймаутов** для защиты от "зависших" моделей:

- **`_STREAM_FIRST_CHUNK_TIMEOUT = 30.0` секунд.** Если за это время модель не прислала ни одного чанка, считается, что она зависла, и выбрасывается `asyncio.TimeoutError`.
- **`_STREAM_CHUNK_TIMEOUT = 15.0` секунд.** Если модель начала отвечать, но между чанками прошло больше этого времени, также выбрасывается таймаут.

Эта система гарантирует, что пользователь не будет ждать ответа вечность, даже если MWS API работает нестабильно.

### 4. Что видит пользователь при разных сценариях

| Сценарий | Поведение |
|---|---|
| Основная модель отвечает успешно | Пользователь видит стриминг ответа. Никаких сообщений о fallback. |
| Основная модель недоступна (404), fallback работает | Пользователь видит сообщение `⚠ Модель 'xxx' недоступна, переключаюсь на резервную...`, затем ответ от fallback-модели. |
| Основная модель зависла (таймаут), fallback работает | То же самое, но с сообщением о таймауте. |
| Все модели недоступны | Пользователь видит `Сервис временно недоступен. Попробуйте повторить запрос через несколько минут.` |
| MWS API вернул `401 Unauthorized` | Это исключение также будет перехвачено, и цепочка пойдёт дальше. Однако если `MWS_API_KEY` неверен, fallback тоже не сработает, и пользователь увидит финальную ошибку. |

### 5. Важные нюансы

- **Технические детали ошибок НЕ показываются пользователю.** Пользователь никогда не увидит "MWS API вернул 500" или "Connection refused". Это сделано для безопасности и чистоты UX.
- **Все ошибки логируются.** В логах контейнера `gpthub_pipelines` можно увидеть полную трассировку, включая оригинальное сообщение от MWS API.
- **`asyncio.CancelledError` не перехватывается.** Если клиент (OpenWebUI) закрыл соединение, стрим корректно прерывается, и это не считается ошибкой.

---

## Часть 10: Альтернативная веб-панель `workspace.html`

В корне `pipelines/static/` лежит файл `workspace.html`. Это полностью самодостаточная HTML/JS-панель для управления git-системой, которая **не зависит от OpenWebUI**.

### 1. Зачем она нужна?

1. **Отладка.** Если что-то не работает в интеграции с OpenWebUI, можно открыть эту панель и проверить, работает ли API напрямую.
2. **Автономность.** Её можно использовать даже если OpenWebUI не запущен или недоступен.
3. **Демонстрация.** Показывает, что вся логика git-системы находится в пайплайнах, а UI — это просто клиент.

### 2. Как она работает?

- **Доступ:** `http://localhost:9099/workspace`
- **Технологии:** Чистый HTML, CSS и JavaScript. Никаких фреймворков, сборщиков или зависимостей.
- **API:** Напрямую вызывает эндпоинты пайплайнов (`/api/v1/workspace/*` и `/api/v1/profile/*`), используя `fetch`.

### 3. Ключевые отличия от Svelte-интерфейса

| Характеристика | `workspace.html` | Svelte-интерфейс |
|---|---|---|

| Характеристика | `workspace.html` | Svelte-интерфейс |
|---|---|---|
| **Аутентификация** | Ручной ввод `User ID` в поле. | Автоматически берёт `user.id` из стора OpenWebUI. |
| **Роутинг** | Вкладки переключаются через JavaScript, без изменения URL. | Использует Svelte-сторы для управления режимами. |
| **Стилизация** | Полностью кастомные CSS-стили в одном файле. | Использует Tailwind CSS и компоненты OpenWebUI. |
| **Сборка** | Не требует сборки, работает "как есть". | Требует `npm run build`. |

### 4. Структура `workspace.html`

Файл представляет собой один большой HTML-документ, внутри которого:

- **`<style>`** — все стили приложения (~600 строк).
- **`<body>`** — разметка интерфейса:
  - `.header` — заголовок и переключатель режимов.
  - `.auth-bar` / `.profile-bar` — поля для ввода `User ID`.
  - `.sidebar` — список чатов или workspaces.
  - `.main` — основная область с деталями.
- **`<script>`** — вся логика приложения (~1000 строк JavaScript):
  - Объект `state` — глобальное состояние.
  - Объект `API` — функции для вызова эндпоинтов.
  - Функции рендеринга (`renderChatsList`, `renderCommits`, ...).
  - Обработчики событий.

### 5. Важные ограничения

- **Нет реактивности.** После каждого действия (загрузка, удаление) нужно **явно вызывать функции рендеринга**.
- **Нет роутинга.** Нельзя поделиться ссылкой на конкретный чат или workspace.
- **Нет валидации `User ID`.** Любая строка считается валидным ID.

### 6. Как используется в разработке

1. Запускаем только `docker compose up pipelines chromadb postgresql searxng` (без OpenWebUI).
2. Открываем `http://localhost:9099/workspace`.
3. Вводим тестовый `User ID` (можно найти в логах пайплайнов).
4. Тестируем создание share-кодов, форки, workspaces.

## Часть 11: Пошаговый сценарий "Пользователь сделал `/fork`"

Этот сценарий покажет полную цепочку вызовов от команды в чате до появления контекста в ответе модели. Это самый сложный и показательный кейс, демонстрирующий взаимодействие всех компонентов.

### Шаг 0: Предварительные условия

- Пользователь **А** ранее создал share-код `BLUE-TIGER-42` с помощью `/share`.
- В базе данных SQLite есть запись в `share_codes` и несколько коммитов в `context_commits`.
- Пользователь **Б** находится в **своём** чате (chat_id = `chat-bbb`).

### Шаг 1: Пользователь Б вводит команду

**Действие:** Пользователь Б пишет в чат OpenWebUI:

```
/fork BLUE-TIGER-42
```

### Шаг 2: Запрос уходит в пайплайны

1. OpenWebUI отправляет POST-запрос на `http://pipelines:9099/v1/chat/completions`.
2. Тело запроса (`body`) содержит:
   - `messages`: `[{"role": "user", "content": "/fork BLUE-TIGER-42"}]`
   - `model`: `"gpthub-auto"`
   - Заголовки `x-openwebui-user-id`, `x-openwebui-chat-id`.

### Шаг 3: `main.py` принимает запрос

1. Эндпоинт `/v1/chat/completions` вызывает `_extract_user_info(request)`.

2. Из заголовков извлекается `user_id` = `"user-bbb"` и `chat_id` = `"chat-bbb"`.
3. Эти данные добавляются в `body`:

```
body["_user_id_resolved"] = "user-bbb"
body["_user_info"] = {"id": "user-bbb", "chat_id": "chat-bbb", ...}
```

4. Вызывается `router.route(body)`.

### Шаг 4: `SmartRouter.route()` анализирует запрос

1. `TaskDetector.detect()` видит, что текст начинается с `/fork`.
2. `_is_git_command()` возвращает `(True, "/fork")`.
3. `DetectionResult`:
    - `task_type`: `TaskType.CHAT_GIT`
    - `detected_model`: `cfg.model_safe_fallback`
    - `is_auto`: `True`
4. В `body` добавляется ключ `_gpthub` с метаданными.
5. Выбирается пайплайн `ChatGitPipeline`.
6. Вызывается `await chat_git_pipeline.run(body)`.

### Шаг 5: `ChatGitPipeline.run()` обрабатывает команду

1. `meta = self.get_meta(body)` → извлекает `user_id` и `chat_id`.
2. `command = meta.get("last_user_message")` → `"/fork BLUE-TIGER-42"`.
3. Вызывается `_handle_fork(command, chat_id, user_id, storage, context)`.

### Шаг 6: `_handle_fork()` — основная логика

```
# 1. Парсинг кода
parts = command.split()
code = parts[1].upper()  # "BLUE-TIGER-42"

# 2. Получение данных кода из SQLite
share_data = await storage.get_share_code(code)
# share_data = {
#     "chat_id": "chat-aaa",
#     "created_by": "user-aaa",
#     "commit_ids": '["commit-1", "commit-2", "commit-3"]',
#     ...
# }

# 3. Получение контекста через ContextManager
source_context = await context.get_context_for_fork(
    source_chat_id = share_data["chat_id"],  # "chat-aaa"
    commit_ids     = json.loads(share_data["commit_ids"]),
    max_tokens     = 600,
)
```

### Шаг 7: `ContextManager.get_context_for_fork()` собирает контекст

1. `storage.get_commits("chat-aaa")` → получает все коммиты чата A.
2. Фильтрует, оставляя только те, что в `commit_ids`.
3. Группирует по `pipeline_type`.
4. Для каждого коммита вызывает `_format_commit_for_fork()` с лимитом символов.
5. Собирает всё в одну строку с разделителями `---`.
6. Обрезает до `_TOTAL_CONTEXT_LIMIT` (3000 символов).
7. Возвращает строку `source_context`.

### Шаг 8: Возврат в `_handle_fork()` — импорт контекста

```
# 4. Импорт контекста в чат Б
imported = await context.import_context(
    target_chat_id = chat_id,        # "chat-bbb"
    target_user_id = user_id,        # "user-bbb"
    source_context = source_context,
    share_code     = code,
    source_chat_id = share_data["chat_id"],
)
```

### Шаг 9: `ContextManager.import_context()` сохраняет контекст

1. Создаёт эмбеддинг для `source_context`.
2. Сохраняет его в коллекцию `branch_chat-bbb` в ChromaDB с метаданными:
    - `pipeline_type`: `"fork"`
    - `source`: `"fork"`
    - `share_code`: `"BLUE-TIGER-42"`
3. Возвращает `1` (успешно импортирован).

### Шаг 10: Завершение `_handle_fork()`

```
# 5. Сохранение связи форка в SQLite
await storage.save_fork_link(
    child_chat_id  = chat_id,        # "chat-bbb"
    parent_chat_id = share_data["chat_id"],  # "chat-aaa"
    share_code     = code,
)

# 6. Инкремент счётчика использований кода
await storage.mark_code_used(code, user_id)

# 7. Формирование и стриминг ответа пользователю
yield "□ **Контекст успешно импортирован!**\n\n"
yield f"```\n{source_context[:400]}\n```\n\n"
```

### Шаг 11: Пользователь Б продолжает диалог

1. Пользователь Б пишет следующий вопрос, например, "А что там было про бюджет?"
2. Запрос снова проходит через `SmartRouter` и попадает в `TextDefaultPipeline`.
3. **Важно:** При формировании контекста для этого запроса `ContextManager.search_context()` найдёт в ChromaDB импортированный форк-контекст, так как он семантически близок к вопросу.
4. Модель ответит, **учитывая информацию из форка**.

### Визуализация цепочки вызовов

```
Пользователь Б
    │
    ▼
/main.py /v1/chat/completions
    │
    ▼
SmartRouter.route()
    │
    ├─► TaskDetector.detect() → TaskType.CHAT_GIT
    │
    ▼
ChatGitPipeline.run()
    │
    ▼
_handle_fork()
    │
    ├─► BranchStorage.get_share_code()          → SQLite
    ├─► ContextManager.get_context_for_fork()   → SQLite + ChromaDB (read)
    ├─► ContextManager.import_context()         → ChromaDB (write)
    ├─► BranchStorage.save_fork_link()          → SQLite
    └─► BranchStorage.mark_code_used()          → SQLite
    │
    ▼
Ответ пользователю Б: "□ Контекст успешно импортирован!"
```

---

## Самопроверка полноты (третья итерация)

Теперь документация покрывает:

1. Архитектуру и структуру проекта.
2. Конфигурацию и все переменные окружения.
3. Принцип работы `SmartRouter` и `TaskDetector` с приоритетами.
4. Детали реализации каждого пайплайна.
5. Работу git-системы: SQLite, ChromaDB, сжатие контекста.
6. Фронтенд: сторы, компоненты, взаимодействие.
7. Критические точки: инъекция метаданных, проксирование, реактивность.
8. Форматирование контекста для форка.
9. **Обработку ошибок MWS API и fallback-стратегии.**
10. **Альтернативную веб-панель `workspace.html`.**
11. **Пошаговый сценарий `/fork` от начала до конца.**

Отлично. Продолжаю дополнять документацию последними четырьмя пунктами. Это будет финальный штрих, который сделает её по-настоящему полной.

---

# Документация GPTHub v2.0.0 (Продолжение)

## Часть 12: Пошаговый сценарий "Пользователь сделал `/share`"

Этот сценарий показывает, как создаётся share-код и как коммиты сохраняются в историю.

### Шаг 0: Предварительные условия

- Пользователь **А** находится в чате `chat-aaa`.
- В этом чате уже есть несколько сохранённых коммитов (от предыдущих ответов модели).
- Пользователь хочет поделиться последними 5 записями.

### Шаг 1: Пользователь А вводит команду

**Действие:** Пользователь А пишет в чат OpenWebUI:

```
/share 5 --alias myproject
```

Флаги:

- `5` — взять последние 5 коммитов.
- `--alias myproject` — задать кастомное имя кода вместо случайного.

## Шаг 2: Запрос проходит через цепочку до `ChatGitPipeline`

Как и в случае с `/fork`, запрос проходит через `main.py` → `SmartRouter` → `ChatGitPipeline.run()`.

## Шаг 3: `_handle_share()` — основная логика

Файл `pipelines/pipelines/chat_git.py`, метод `_handle_share()`.

**3.1. Парсинг команды**

```
parts     = command.split()
flags     = set(p.lower() for p in parts if p.startswith("--"))
non_flags = [p for p in parts[1:] if not p.startswith("--")]

is_permanent = "--permanent" in flags

alias = None
if "--alias" in flags:
    # Извлекаем значение после --alias
    for i, p in enumerate(parts):
        if p.lower() == "--alias" and i + 1 < len(parts):
            alias = parts[i + 1]
            break

limit = 10  # дефолт
for p in non_flags:
    if p.isdigit():
        limit = min(int(p), 50)
        break
```

**3.2. Валидация alias**

```
if alias is not None:
    alias = _normalize_alias(alias)
    error = _validate_alias(alias)
    if error:
        yield f"□ {error}\n"
        return
```

- `_validate_alias()` проверяет, что alias содержит только латиницу, цифры, дефис и подчёркивание, длина 3-32 символа.

**3.3. Получение коммитов из SQLite**

```
commits = await storage.get_commits(chat_id, limit=limit)
if not commits:
    yield "⚠ В этом чате пока нет сохранённых контекстов.\n\n"
    return

commit_ids = [c["id"] for c in commits]
```

**3.4. Создание share-кода в SQLite**

```
code = await storage.create_share_code(
    chat_id      = chat_id,
    user_id      = user_id,
    commit_ids   = commit_ids,
    is_permanent = is_permanent,
    alias        = alias,
)
```

**3.5. Что происходит внутри `BranchStorage.create_share_code()`**

Файл `pipelines/chat_branches/storage.py`.

```
def _create_share_code_sync(...):
    now = int(time.time())

    if is_permanent:
        expires = now + 100 * 365 * 24 * 3600  # 100 лет
        # Проверка rate-limit
        window_start = now - _PERMANENT_RATE_WINDOW
        count = conn.execute(
            "SELECT COUNT(*) FROM share_codes WHERE created_by = ? AND is_permanent = 1 AND created_at > ?",
            (user_id, window_start)
        ).fetchone()[0]
        if count >= _PERMANENT_RATE_LIMIT:
            raise ValueError("Слишком много постоянных кодов. Максимум 20 в час.")
    else:
        expires = now + 48 * 3600  # 48 часов

    # Если alias задан — удаляем старый код с таким же alias
    if alias:
        conn.execute("DELETE FROM share_codes WHERE alias = ?", (alias,))
        code = alias
    else:
        code = _generate_unique_code(conn)  # например, "BLUE-TIGER-42"

    # Вставка записи
    conn.execute("""
        INSERT INTO share_codes
        (code, chat_id, created_by, created_at, expires_at, is_permanent, alias, use_count, commit_ids)
        VALUES (?, ?, ?, ?, ?, ?, ?, 0, ?)
    """, (code, chat_id, user_id, now, expires, int(is_permanent), alias, json.dumps(commit_ids)))
```

**3.6. Формирование ответа пользователю**

```
yield "□ **Share Code создан!**\n\n"
yield f"```\n{code}\n```\n\n"
yield f"**Включено записей:** {len(commit_ids)}\n"

if is_permanent:
    yield "**Срок действия:** ♾ Постоянный (многоразовый)\n"
else:
    yield "**Срок действия:** 48 часов (одноразовый)\n"

if alias:
    yield f"**Alias:** `{code}`\n"

yield f"\nДругой пользователь пишет:\n```\n/fork {code}\n```\n"
```

## Шаг 4: Что происходит в фоне при обычных ответах модели (как создаются коммиты)

Чтобы `/share` было чем делиться, коммиты должны были создаться ранее. Это происходит **автоматически** после каждого ответа модели.

**Цепочка вызовов:**

1. Любой пайплайн (например, `TextDefaultPipeline`) завершает стриминг ответа.
2. Вызывает `self.save_chat_context(...)` через `fire_and_forget`.
3. `BasePipeline.save_chat_context()` вызывает `ContextManager.compress_and_store()`.
4. `ContextManager.compress_and_store()`:
   - Генерирует `summary` через LLM.
   - Генерирует `tags` через LLM.
   - Сохраняет summary в ChromaDB (коллекция `branch_{chat_id}`).
   - Вызывает `BranchStorage.save_commit()`.
5. `BranchStorage.save_commit()` вставляет запись в `context_commits`.

**Запись в `context_commits`:**

```
INSERT INTO context_commits
(id, chat_id, user_id, timestamp, pipeline_type, user_message, assistant_summary, tags, tokens_count, parent_id)
VALUES (
    'uuid-xxxx',           -- id
    'chat-aaa',            -- chat_id
    'user-aaa',            -- user_id
    1712345678,            -- timestamp
    'text',                -- pipeline_type
    'Как дела?',           -- user_message
    'Пользователь спросил о состоянии дел. Ответ: всё хорошо.', -- assistant_summary
    '["приветствие"]',     -- tags
    150,                   -- tokens_count
    NULL                   -- parent_id
);
```

Таким образом, к моменту вызова `/share` в базе уже есть коммиты, готовые к экспорту.

---

# Часть 13: Пошаговый сценарий "Создание и вступление в Workspace"

Этот сценарий показывает, как устроена командная работа через workspaces.

## Сценарий A: Владелец создаёт workspace и приглашает участника

**Шаг 1: Владелец создаёт workspace**

**Действие:** Владелец пишет в чат:

```
/branch create Разработка GPTHub
```

**Обработка в `ChatGitPipeline._handle_branch_create()`:**

```
ws_id = str(uuid.uuid4())[:8]  # например, "a1b2c3d4"
ws = await storage.create_workspace(
    workspace_id = ws_id,
    name         = "Разработка GPTHub",
    owner_id     = user_id,
)
```

**Что происходит в `BranchStorage.create_workspace()`:**

```
-- Вставка в таблицу workspaces
INSERT INTO workspaces (id, name, owner_id, created_at)
VALUES ('a1b2c3d4', 'Разработка GPTHub', 'user-aaa', 1712345678);

-- Вставка владельца как участника
INSERT INTO workspace_members (workspace_id, user_id, role, joined_at)
VALUES ('a1b2c3d4', 'user-aaa', 'owner', 1712345678);
```

**Ответ владельцу:**

```
 **Workspace создан!**

**Название:** Разработка GPTHub
**ID:** `a1b2c3d4`

**Следующий шаг — пригласи команду:**
```

/branch invite a1b2c3d4

**Шаг 2: Владелец создаёт invite-код**

**Действие:** Владелец пишет:

```
/branch invite a1b2c3d4 --multi
```

**Обработка в `ChatGitPipeline._handle_branch_invite()`:**

```
ws_id = parts[2]  # "a1b2c3d4"
is_multi = "--multi" in parts
max_uses = 100 if is_multi else 1

invite = await storage.create_invite(
    workspace_id = ws_id,
    created_by   = user_id,
    ttl_hours    = 72,
    max_uses     = max_uses,
)
```

**Что происходит в `BranchStorage.create_invite()`:**

1. Проверка прав: `SELECT role FROM workspace_members WHERE workspace_id = ? AND user_id = ?`. Только `owner` или `admin` могут создавать инвайты.
2. Генерация уникального кода: `INV-AB12-CD34`.
3. Вставка записи:

```
INSERT INTO workspace_invites
(id, workspace_id, invite_code, created_by, created_at, expires_at, max_uses, use_count)
VALUES (
    'invite-uuid',
    'a1b2c3d4',
    'INV-AB12-CD34',
    'user-aaa',
    1712345678,
    1712604878,  -- +72 часа
    100,
    0
);
```

**Ответ владельцу:**

```
 **Invite-код создан!**

**Workspace:** Разработка GPTHub
```

INV-AB12-CD34

```
**Тип:** Многоразовый (до 100 человек)
**Действует до:** 08.04.2026 15:34

**Коллега пишет:**
```

/branch join INV-AB12-CD34

## Сценарий Б: Участник вступает в workspace

**Шаг 1: Участник вводит команду**

**Действие:** Участник Б пишет в чат:

```
/branch join INV-AB12-CD34
```

**Обработка в `ChatGitPipeline._handle_branch_join()`:**

```
invite_code = parts[2].upper()  # "INV-AB12-CD34"
result = await storage.join_workspace(
    invite_code = invite_code,
    user_id     = user_id,
)
```

**Что происходит в `BranchStorage.join_workspace()`:**

```
-- 1. Поиск инвайта
SELECT * FROM workspace_invites
WHERE invite_code = 'INV-AB12-CD34'
  AND expires_at > 1712345678;

-- 2. Проверка лимита использований
-- (use_count < max_uses)

-- 3. Проверка, что пользователь ещё не в workspace
SELECT user_id FROM workspace_members
WHERE workspace_id = 'a1b2c3d4' AND user_id = 'user-bbb';
-- (должен вернуть пустой результат)

-- 4. Добавление участника
INSERT INTO workspace_members
(workspace_id, user_id, role, joined_at, invited_by)
VALUES ('a1b2c3d4', 'user-bbb', 'member', 1712345678, 'user-aaa');

-- 5. Инкремент счётчика использований
UPDATE workspace_invites
SET use_count = use_count + 1
WHERE id = 'invite-uuid';
```

**Ответ участнику:**

```
 **Ты вступил в workspace!**

**Workspace:** Разработка GPTHub
**ID:** `a1b2c3d4`
**Роль:** `member`

 Просмотри список участников:
```

/branch members a1b2c3d4

### Шаг 2: Участник смотрит список участников

**Действие:**

```
/branch members a1b2c3d4
```

**Обработка в `ChatGitPipeline._handle_branch_members()`:**

```
members = await storage.get_workspace_members(
    workspace_id       = ws_id,
    requesting_user_id = user_id,
)
```

**Что происходит в `BranchStorage.get_workspace_members()`:**

1. Проверка, что запрашивающий — участник workspace.
2. Запрос:

```
SELECT user_id, role, joined_at, invited_by
FROM workspace_members
WHERE workspace_id = 'a1b2c3d4'
ORDER BY
    CASE role
        WHEN 'owner' THEN 0
        WHEN 'admin' THEN 1
        ELSE 2
    END,
    joined_at ASC;
```

**Ответ:**

```
 **Участники workspace `a1b2c3d4` (2):**

 `user-aaa...`
   Роль: `owner` · вступил 01.04.2026

 `user-bbb...`
   Роль: `member` · вступил 03.04.2026
   Приглашён: `user-aaa...`
```

## Часть 14: Детальный разбор миграций SQLite

Файл `pipelines/chat_branches/storage.py`, функция `_run_migrations()`.

GPTHub использует простую, но надёжную систему миграций "на лету". При каждом запуске `BranchStorage` проверяет схему базы данных и добавляет недостающие колонки или таблицы.

### Как это работает

```
def _run_migrations(conn: sqlite3.Connection) -> None:
    # 1. Проверяем наличие колонки pipeline_type в context_commits
    existing_cols = {
        row[1] for row in
        conn.execute("PRAGMA table_info(context_commits)").fetchall()
    }
    if "pipeline_type" not in existing_cols:
        conn.execute(
            "ALTER TABLE context_commits "
            "ADD COLUMN pipeline_type TEXT NOT NULL DEFAULT 'text'"
        )
        logger.info("migration: добавлен pipeline_type в context_commits")

    # 2. Проверяем наличие колонки is_permanent в share_codes
    share_cols = {
        row[1] for row in
        conn.execute("PRAGMA table_info(share_codes)").fetchall()
    }
    if "is_permanent" not in share_cols:
        conn.execute(
            "ALTER TABLE share_codes "
            "ADD COLUMN is_permanent INTEGER NOT NULL DEFAULT 0"
        )
        logger.info("migration: добавлен is_permanent в share_codes")

    # 3. Проверяем alias
    if "alias" not in share_cols:
        conn.execute("ALTER TABLE share_codes ADD COLUMN alias TEXT")
        conn.execute("CREATE UNIQUE INDEX IF NOT EXISTS idx_share_alias ON share_codes(alias)")
        logger.info("migration: добавлен alias в share_codes")

    # 4. Проверяем use_count
    if "use_count" not in share_cols:
        conn.execute(
            "ALTER TABLE share_codes "
            "ADD COLUMN use_count INTEGER NOT NULL DEFAULT 0"
        )
        logger.info("migration: добавлен use_count в share_codes")

    # 5. Проверяем invited_by в workspace_members
    ws_member_cols = {
        row[1] for row in
        conn.execute("PRAGMA table_info(workspace_members)").fetchall()
    }
    if "invited_by" not in ws_member_cols:
        conn.execute(
            "ALTER TABLE workspace_members ADD COLUMN invited_by TEXT"
        )
        logger.info("migration: добавлен invited_by в workspace_members")

    # 6. Создаём таблицу workspace_invites если её нет
    conn.execute("""
        CREATE TABLE IF NOT EXISTS workspace_invites (
            id            TEXT PRIMARY KEY,
            workspace_id  TEXT NOT NULL,
            invite_code   TEXT NOT NULL UNIQUE,
            created_by    TEXT NOT NULL,
            created_at    INTEGER NOT NULL,
            expires_at    INTEGER NOT NULL,
            max_uses      INTEGER NOT NULL DEFAULT 1,
            use_count     INTEGER NOT NULL DEFAULT 0
        )
```

```
    """)
    # ... создание индексов
```

Почему такой подход выбран?

| Плюсы | Минусы |
|---|---|
| **Простота.** Не требует внешних инструментов (Alembic, Goose). | Нет версионирования. Нельзя откатить миграцию. |
| **Автоматичность.** При первом запуске всё создаётся само. | Нет явного файла с историей изменений схемы. |
| **Надёжность.** `IF NOT EXISTS` и проверки через `PRAGMA` гарантируют идемпотентность. | Сложнее отследить, какая версия схемы сейчас в проде. |

Как добавить новую миграцию

1. Добавить блок проверки в `_run_migrations()`.
2. Использовать `PRAGMA table_info(...)` для проверки существования колонки/таблицы.
3. Выполнить `ALTER TABLE ... ADD COLUMN ...` или `CREATE TABLE IF NOT EXISTS ...`.
4. Добавить логирование.

**Пример добавления колонки `priority` в `context_commits`:**

```
existing_cols = {row[1] for row in conn.execute("PRAGMA table_info(context_commits)")}
if "priority" not in existing_cols:
    conn.execute("ALTER TABLE context_commits ADD COLUMN priority INTEGER NOT NULL DEFAULT 0")
    logger.info("migration: добавлен priority в context_commits")
```

---

# Часть 15: Справочник всех команд `ChatGitPipeline`

Этот раздел — как `man` для git-системы GPTHub. Полный список всех доступных команд с синтаксисом, флагами и примерами.

## `/share` — Поделиться контекстом чата

| Элемент | Описание |
|---|---|
| **Синтаксис** | `/share [количество] [флаги]` |
| **Аргументы** | `количество` — число последних коммитов (по умолчанию 10, максимум 50) |
| **Флаги** | `--permanent` — постоянный многоразовый код<br>`--alias ИМЯ` — кастомное имя кода (латиница, цифры, -, _, 3-32 символа)<br>`--list` — показать все активные коды<br>`--revoke КОД` — отозвать код<br>`--public` — сделать чат публичным (виден в профиле)<br>`--private` — скрыть чат из профиля |
| **Примеры** | `/share` — поделиться 10 последними записями<br>`/share 5` — поделиться 5 последними записями<br>`/share --permanent` — создать постоянный код<br>`/share --alias myproject` — код с именем `MYPROJECT`<br>`/share --list` — список кодов<br>`/share --revoke BLUE-TIGER-42` — отозвать код<br>`/share --public` — опубликовать чат |

## `/fork` — Импортировать контекст из share-кода

| Элемент | Описание |
|---|---|
| **Синтаксис** | `/fork КОД` |
| **Примеры** | `/fork BLUE-TIGER-42` — импортировать все записи из кода<br>`/fork MYPROJECT` — импортировать по alias |

## `/cherry-pick` — Выборочный импорт записей

| Элемент | Описание |
|---|---|
| **Синтаксис** | `/cherry-pick КОД [флаги] [номера]` |
| **Аргументы** | `номера` — номера записей для импорта (нумерация с 1) |
| **Флаги** | `--preview` — показать список записей без импорта<br>`--full` — импортировать без сжатия (полный текст) |
| **Примеры** | `/cherry-pick BLUE-TIGER-42 --preview` — посмотреть что внутри<br>`/cherry-pick BLUE-TIGER-42 1 3` — взять записи 1 и 3<br>`/cherry-pick BLUE-TIGER-42 2 --full` — запись 2 без сжатия |

## `/log` — История контекстов текущего чата

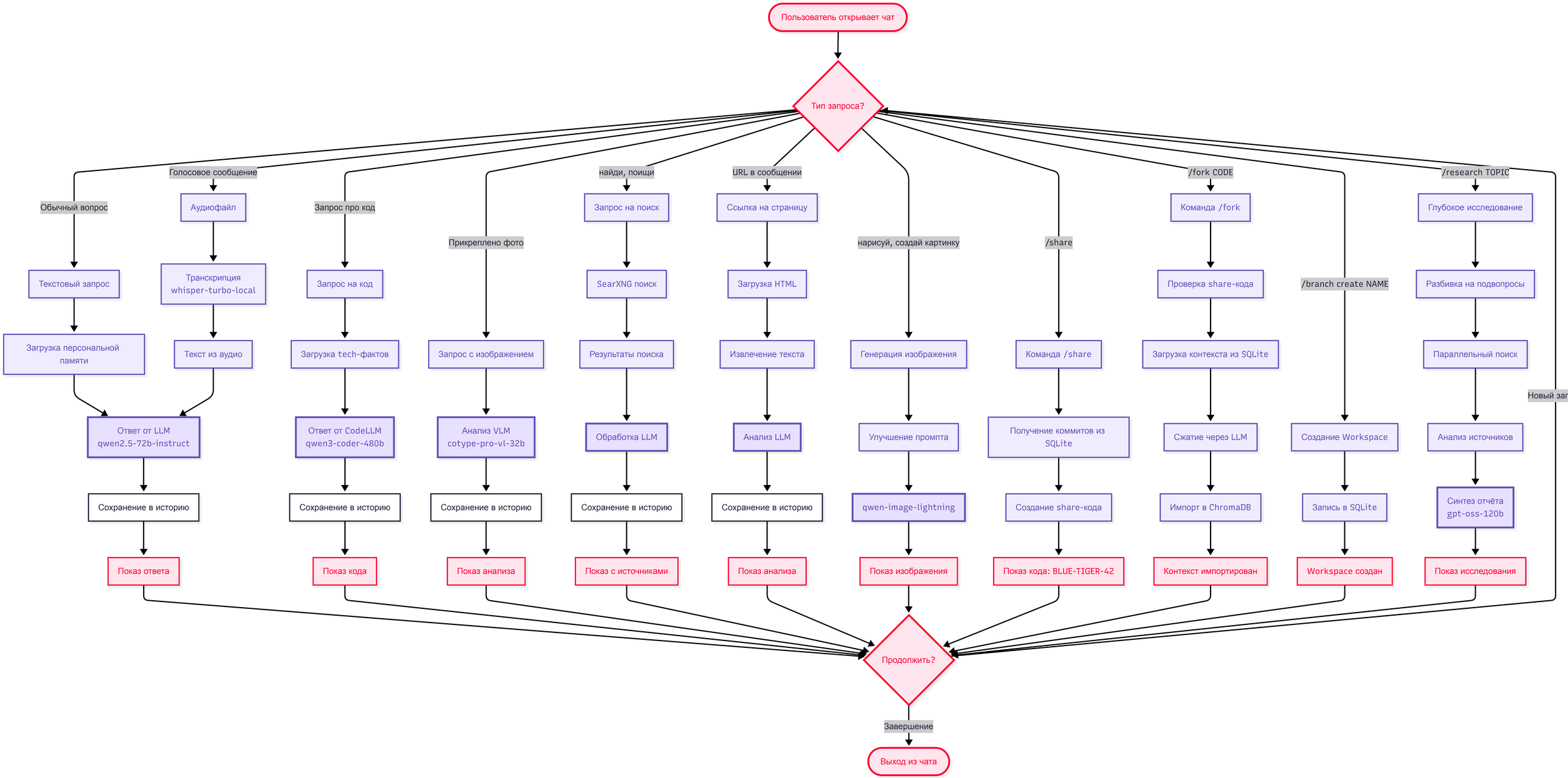
| Элемент | Описание |
|---|---|
| **Синтаксис** | `/log` |
| **Примеры** | `/log` — показать последние 20 записей с датами и тегами |

`/branch` — Управление workspaces

| Подкоманда | Синтаксис | Описание | Пример |
|---|---|---|---|
| `list` | `/branch list` | Показать все workspaces пользователя | `/branch list` |
| `create` | `/branch create ИМЯ` | Создать новый workspace | `/branch create Разработка GPTHub` |
| `join` | `/branch join КОД` | Вступить в workspace по invite-коду | `/branch join INV-AB12-CD34` |
| `invite` | `/branch invite WS_ID [--multi] [--hours N]` | Создать invite-код | `/branch invite a1b2c3d4 --multi --hours 168` |
| `members` | `/branch members WS_ID` | Показать участников workspace | `/branch members a1b2c3d4` |
| `chats` | `/branch chats [WS_ID]` | Показать публичные чаты (свои или workspace) | `/branch chats` |
| `help` | `/branch help` | Показать справку | `/branch help` |

**Флаги для `invite`:**

- `--multi` — многоразовый код (до 100 использований).
- `--hours N` — срок действия в часах (по умолчанию 72, максимум 720).


Пользователь открывает чат
Тип запроса?
Обычный вопрос
Текстовый запрос
Загрузка персональной памяти
Голосовое сообщение
Аудиофайл
Транскрипция whisper-turbo-local
Текст из аудио
Ответ от LLM qwen2.5-72b-instruct
Сохранение в историю
Показ ответа
Запрос про код
Запрос на код
Загрузка tech-фактов
Ответ от CodeLLM qwen3-coder-480b
Сохранение в историю
Показ кода
Прикреплено фото
Запрос с изображением
Анализ VLM cotype-pro-vl-32b
Сохранение в историю
Показ анализа
найди, поищи
Запрос на поиск
SearXNG поиск
Результаты поиска
Обработка LLM
Сохранение в историю
Показ с источниками
URL в сообщении
Ссылка на страницу
Загрузка HTML
Извлечение текста
Анализ LLM
Сохранение в историю
Показ анализа
нарисуй, создай картинку
Генерация изображения
Улучшение промпта
qwen-image-lightning
Показ изображения
/share
Команда /share
Получение коммитов из SQLite
Создание share-кода
Показ кода: BLUE-TIGER-42
/fork CODE
Команда /fork
Проверка share-кода
Загрузка контекста из SQLite
Сжатие через LLM
Импорт в ChromaDB
Контекст импортирован
/branch create NAME
Создание Workspace
Запись в SQLite
Workspace создан
/research TOPIC
Глубокое исследование
Разбивка на подвопросы
Параллельный поиск
Анализ источников
Синтез отчёта gpt-oss-120b
Показ исследования
Новый запрос
Продолжить?
Завершение
Выход из чата